10

Building an AI Agent Application

In the previous chapter, we discussed how an LLM can extend its capabilities by using other tools. We also saw some examples of how the use of multiple agents at the same time (instead of one) can be used to solve more complex tasks. We extensively discussed how these approaches can be used in various industries and how they can be revolutionary for so many applications. However, we also highlighted two of the limitations of agents: scalability and the complexity of connecting an agent with different tools.

In this chapter, we will expand on these challenges and show how we can overcome them. We will pick up from these two limitations. So far, we have treated multi-agent systems as standalone entities running on a personal computer. In the final section of the previous chapter, we explored the exciting new business paradigms emerging with AI. Agents are poised to play a significant role across industries in the future, but for that to happen, agent systems must be ready for production deployment. Getting a multi-agent system into production means we'll have to solve the previously mentioned scalability and complexity issues to avoid harming the customer experience.

We will follow a progressive approach in this chapter. We will use Streamlit, which is a simple but flexible framework that allows us to manage the entire process of creating an application around our agents. It allows us to conduct rapid prototyping of our application, testing different options until we reach a proof of concept. With Streamlit, we can seamlessly work with both the backend, where agents operate, and the frontend, which shapes the user experience—all within a single framework.

Next, we will discuss in more detail the whole set of operations that are necessary to make an LLM and agents functional. Irrespective of whether you have the opportunity to train a model from scratch, this section will help you understand how to improve scalability and how the industry is handling the complexity of the process. In addition, we will address asynchronous programming and containerization, two concepts that are useful for scaling not only a multi-agent application but any machine learning project.

In this chapter, we'll be covering the following topics:

- Introduction to Streamlit
- Developing our frontend with Streamlit
- Creating an application with Streamlit and AI agents
- Machine learning operations and LLM operations
- Asynchronous programming
- Docker

Technical requirements

Most of the code in this chapter can be run on CPUs. The *Introduction to Streamlit* and *Frontend with Streamlit* sections do not require GPUs. The libraries to install are as follows:

- **Streamlit**: For managing the frontend and backend of our app
- **pandas**: For handling DataFrames
- **Matplotilib**: For plotting graphs
- **Folium**: For plotting maps
- **time**: For monitoring runtime
- **NumPy**: For numerical computation
- **pydeck**: For map representation
- **OpenAI**: For building agents using its LLMs
- **Sentence Transformer**: To conduct embeddings

The *Creating an application with Streamlit and AI agents* section can be run on a CPU, but it would be preferred if it were run on a GPU.

The OpenAI library requires the use of an OpenAI token, and you should register with OpenAI to obtain it. The next sections can be run on CPUs and are mainly based on the use of the AsyncIO library. The code can be found on GitHub: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr10**.

Introduction to Streamlit

If readers are familiar with Streamlit, they can move on to the *Creating an application with Streamlit and AI agents* section directly.

Companies have invested heavily in data science and AI. The models that are trained can guide business decisions and provide different insights. Training a model, using it, and extracting insights requires expertise that not everyone has. A model that is truly useful for a company must provide results that must then be used by other stakeholders as well. For example, when you train a model, it should generate results that are usable by other people. It is possible to create static visualizations of the data (exporting graphs), but they convey only limited information. One could provide information in a Jupyter notebook but not everyone is capable of using such a tool. One option that might allow easier access by others is to create a dashboard or web application.

This is where Streamlit comes in.

Starting with Streamlit

Streamlit is a web application framework that allows one to easily and intuitively create web applications with Python. Its library provides a number of built-in components for both the backend and the frontend. It is also compatible with leading machine learning, graph, and plotting libraries.

The objective of this section is to understand how Streamlit works and how it can be a powerful tool.

One of the advantages of Streamlit is its ease of use and installation. Streamlit can simply be installed from the terminal and is present in Anaconda distributions:

```
pip install streamlit
```

Organizing an app with Streamlit is a simple Python script that typically contains both the backend and the frontend. This script can then be run either locally or in the cloud. For example, **`my_app.py`** should contain within it all the elements to build a web app. In the simplest cases, with just a few lines of code, we can build a web app. Once we define our app, running it locally is really simple:

```
streamlit run my_app.py
```

What we need to do is call Streamlit and the name of our app (obviously, if we are using a terminal, we need to be in the right directory). Actually, the script does not have to be in your local directory; it can be on the internet. For example, our script is in our repository on GitHub, and we want to run it locally:

```
streamlit run https://raw.githubusercontent.com/streamlit/my_apps/master/my_app.py
```

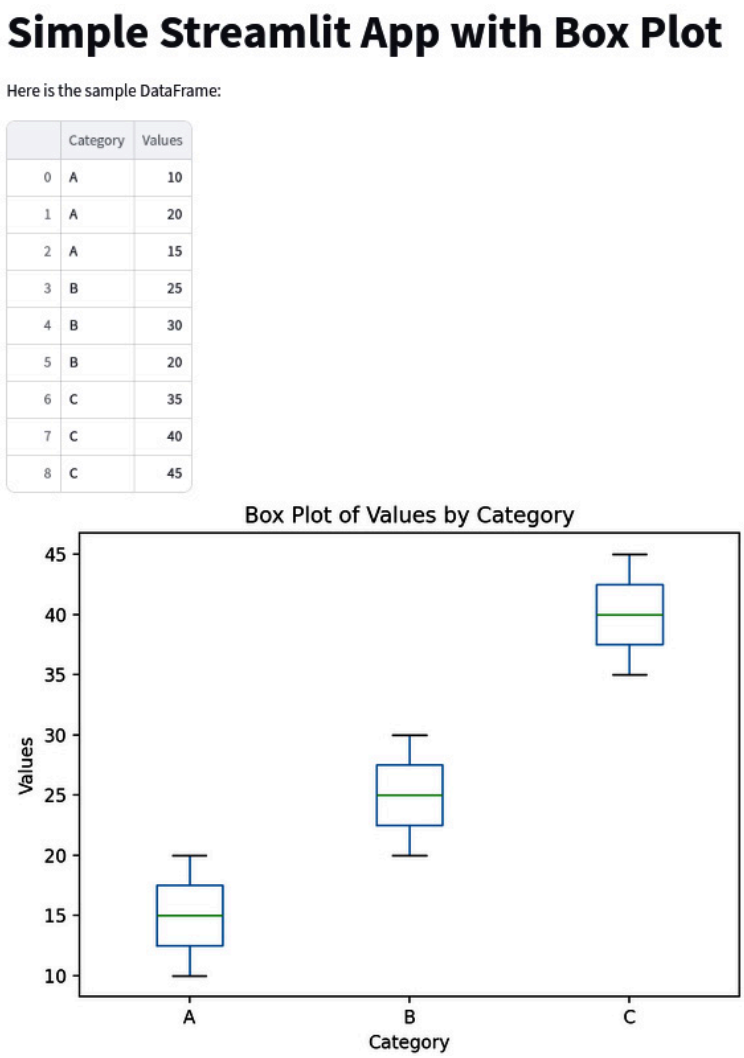
Under the hood, Streamlit runs through the file and executes the elements it finds sequentially. After that is done, a local Streamlit server will be initialized and your app will open in a new tab in your default web browser. Note that everything we write is in Python, and no other language is required. When we make a change, we must save our source. Streamlit detects any modifications and prompts us to rerun the app. This allows for quick iterations while immediately observing the effects, ensuring a seamless feedback loop between writing and running the application.

An example of a simple app is the following:

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
# Title for the app
st.title("Simple Streamlit App with Box Plot")
# Create a sample DataFrame
data = {
    'Category': ['A', 'A', 'A', 'B', 'B', 'B', 'C', 'C', 'C'],
    'Values': [10, 20, 15, 25, 30, 20, 35, 40, 45]
}
df = pd.DataFrame(data)
# Display the DataFrame
st.write("Here is the sample DataFrame:")
st.dataframe(df)
# Create a box plot
fig, ax = plt.subplots()
df.boxplot(column='Values', by='Category', ax=ax, grid=False)
plt.title("Box Plot of Values by Category")
plt.suptitle("")  # Remove the automatic subtitle
plt.xlabel("Category")
plt.ylabel("Values")
```

```
# Display the plot in Streamlit
st.pyplot(fig)
```

The app we generated simply does three things: it creates a DataFrame with pandas, plots it, and then produces a box plot. In a few lines of code, we have created a mini web application that is accessible on our browser. Once we have written it, we just have to run it and then Streamlit takes care of everything else.



	Category	Values
0	A	10
1	A	20
2	A	15
3	B	25
4	B	30
5	B	20
6	C	35
7	C	40
8	C	45

Figure 10.1 – Example of a web application

Let's look at the code block in a bit more detail:

- **`st.title`**: This is a text element that allows us to display the title of our app. It is a good idea to always include it in an app.
- **`st.write`**: This is considered the Swiss army knife of Streamlit. Its main purpose is to write both textual and other elements. In this case, we have shown how passing a DataFrame is written to the app in nice formatting. In addition, this element is interactive. In other words, its behavior depends on the input given to it. The **`write()`** function is not limited to text but can be used with images, other Python elements (such as lists and dictionaries), templates, and so on. It also allows us to insert commands with HTML if we want to edit our text.
- **`st.pyplot`**: This displays a Matplotlib figure – in our case, a box plot. As you can see, we generated our figure first and then called **`pyplot()`** for the subsequent plotting. The figure is generated before being actually shown. In other words, the figure is already present in

memory; we need **`pyplot()`** to display the figure to the user in the app. Actually, we could also call plotting directly with Matplotlib, but this is not recommended because it could lead to unexpected behavior.

Note that we have only shown some basic commands, but Streamlit is quite flexible. For example, the DataFrame can be written to the app in different ways. Using **`st.write()`** is just one way: **`st.dataframe()`** does the same as **`st.write()`**, **`st.table()`** allows us to render the table statically, and writing **`'df'`** directly acts as if we were using **`st.write()`**. It is recommended to use one of the built-in methods because the behavior is known and we can also use additional arguments to handle the output.

For example, we can use the flexibility provided by the built-in method, **`st.dataframe()`**, to highlight elements in our DataFrame:

```
df = pd.DataFrame(data)
st.dataframe(df.style.highlight_max(axis=0))
```

Here is sample DataFrame:

	Category	Values
0	A	10
1	A	20
2	A	15
3	B	25
4	B	30
5	B	20
6	C	35
7	C	40
8	C	45

Figure 10.2 – Change in style in the DataFrame rendering

In addition, Streamlit also makes it easy to add maps to our application. Just provide the coordinates, and **`st.map()`** magically allows us to have a map in our application (a map that we can enlarge and move). In this case, we provided the coordinates of some Sicilian cities:

```
city_data = {
    'City': ['Palermo', 'Syracuse', 'Catania', 'Agrigento'],
    'latitude': [38.1157, 37.0757, 37.5079, 37.2982],
    'longitude': [13.3615, 15.2867, 15.0830, 13.5763]
}
city_data = pd.DataFrame(city_data)
st.map(city_data)
```



Figure 10.3 – Plotting a map with Streamlit

As can be seen, we have added some elements and made some changes to our app (adding a map). Whenever we modify the code, we should remember to save the changes to the script; then, we go to our app and press the *R* key, which will reload the app with the updates.

If there are any errors, Streamlit will provide us with error messages indicating what we need to correct. An example of an error is shown in the following figure (in this case, about the variable name to use):

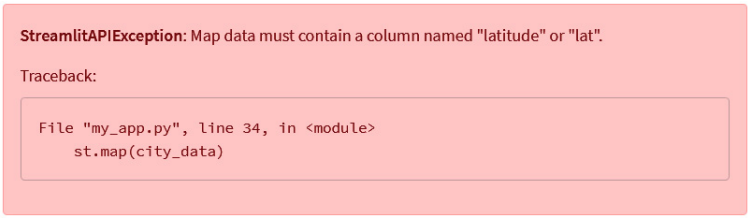


Figure 10.4 – Example of an error

For debugging, we use **`st.write()`** extensively; this simple function can print almost any Python object by guiding us to understand what the error is. For example, we can use it in this case. As we can see, we have an error in the column names (*Latitude* should be lowercase; so, we substitute it with the correct name):

```
st.write(city_data)
```

	City	Latitude	longitude
0	Palermo	38.1157	13.3615
1	Syracuse	37.0757	15.2867
2	Catania	37.5079	15.0830
3	Agrigento	37.2982	13.5763

Figure 10.5 – Using st.write() to debug

Caching the results

Caching allows our app to remain performant even if data is loaded from the web (we will discuss how to add data from the web or the user later).

It also allows it to manipulate large datasets or use machine learning models. So far, we have been using small datasets and hence we could load anything, but what if we start putting models of millions of parameters inside our app? Our app might crash. If we use models or other elements that require long computations, we need to focus on optimizing our app's efficiency by caching results in memory and avoiding redundant calculations. We can see the cache as a kind of short-term memory, where we keep information that we use often or think will be useful to safeguard. Caching allows us to reuse this information and save computation. If we have a function that performs a large computation, we can use two alternatives:

- **`st.cache_data`**: This is a decorator in Streamlit that is used to cache the results of a function so that the function need not be recomputed every time the app is rerun (such as when a user interacts with widgets or the app reloads). This decorator is recommended for cache computations that return data. One should use **`st.cache_data`** when a function returns a serializable data object (e.g., **`str`**, **`int`**, **`float`**, **`DataFrame`**, **`dict`**, or **`list`**). When a function is wrapped with **`@st.cache_data`**, the first time the function is called, Streamlit stores the result in memory or a disk cache, depending on the configuration. On subsequent calls with the same arguments, Streamlit returns the cached result, which is much faster than recomputing it. It speeds up the app by preventing redundant work, especially for functions that take a long time to execute. If the inputs to the function change, Streamlit will invalidate the cache and recompute the function.
- **`st.cache_resource`**: This is another decorator in Streamlit, introduced to handle the caching of resources – specifically, objects or expensive operations that do not depend on the function arguments but instead represent reusable resources that can be cached for the lifetime of the app. While **`st.cache_data`** is used for caching the results of computations or data loads based on the inputs, **`st.cache_resource`** is designed to cache resources such as database connections, model objects, or any other object that is expensive to create or initialize but doesn't change with each function call. Use this for caching resources such as database connections, machine learning models, network connections, or any expensive resource that needs to be reused across multiple runs of the app. If an object or resource (e.g., a pretrained model) is expensive to create, you can use **`st.cache_resource`** to avoid reloading or reinitializing it multiple times.

For example, for **`st.cache_data`**, in the following code, we are simulating a slow operation and showing how caching is saving time:

```
import streamlit as st
import time
# Use @st.cache_data to cache the result
@st.cache_data
def load_data():
    time.sleep(3)  # Simulate a slow operation (e.g., loading a large dataset)
    return "Data loaded!"
# Call the function
st.write(load_data())
in a similar way for st.cache_resource:
import streamlit as st
import time
# Example: A resource-intensive function (e.g., loading a model)
```

```
@st.cache_resource
def load_model():
    time.sleep(5)  # Simulate a slow operation like loading a model
    return "Model loaded!"  # This could be a model object in a real scenario
# Call the function
st.write(load_model())
```

In the preceding snippet, under the hood, before running a function, Streamlit checks its cache for a previously saved result. If it finds one, it uses that instead of running the function; if it doesn't find it, it runs the function and saves it in the cache. The cache is updated during execution, especially if the code changes.

By default, Streamlit doesn't save the information between app reruns, but with each rerun, it reruns the app from top to bottom. Normally, Streamlit reruns the entire script whenever there's an interaction (e.g., when a user adjusts a slider or clicks a button). With session state, you can store data that persists during these reruns so you don't lose values when the script reruns. Each user gets their own independent session state, so data stored in the session state is isolated from other users. You can use the session state to store things such as form inputs, counters, authentication data, or intermediate computation results.

Let's try building an app that makes a shopping list; we will show how to save information about the session:

```
import streamlit as st
# Define a list of grocery items (the initial list of items to buy)
grocery_items = ['Apple', 'Banana', 'Carrot', 'Milk', 'Eggs']
# Streamlit app interface
st.title('Grocery List App')
# Text input to add a new item to the list
new_item = st.text_input("Add a new item to your grocery list:")
# Button to add the new item to the list
if st.button('Add Item'):
    if new_item:
        grocery_items.append(new_item)
        st.success(f"'{new_item}' has been added to your list!")
    else:
        st.warning("Please enter an item to add.")
# Display the current list of grocery items
st.write("### Items to Buy:")
for item in grocery_items:
    st.write(f"- {item}")
```

This is our initial app; we will see immediately afterward how we can view information saved by the user:

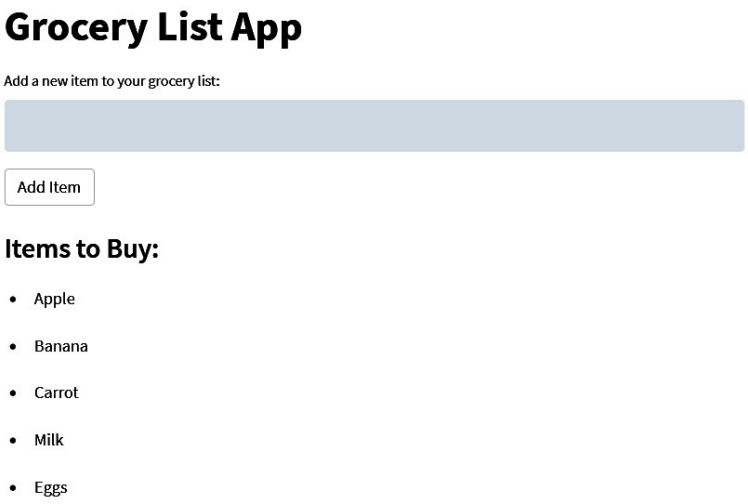


Figure 10.6 – Example of grocery list app

If we add objects by clicking on **Add Item**, they will be added to the list (at this time, the information is not saved; it remains only for the session):

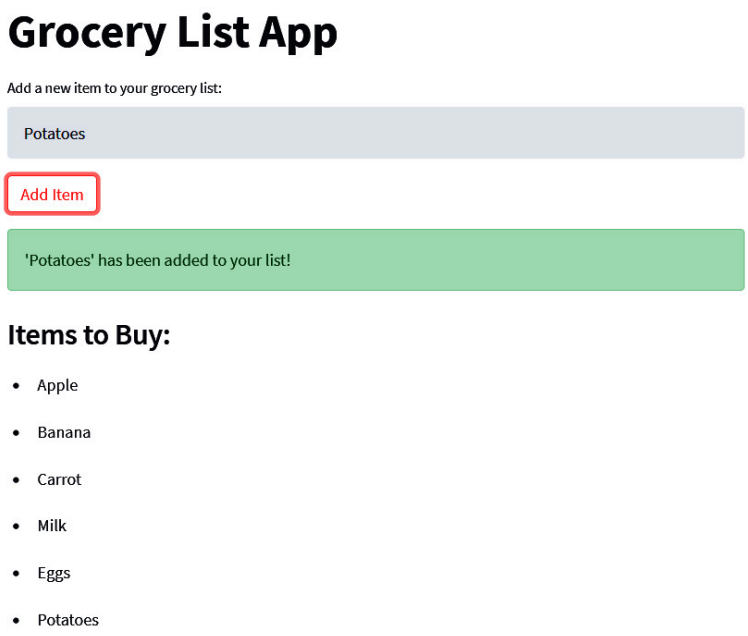


Figure 10.7 – Example of adding objects to the grocery list app

However, if we press *R* and rerun our app, we will lose this information, and the elements will disappear (because the information is not saved anywhere).

Now, let's try **`session_state`**:

```
import streamlit as st
# Initialize session state for grocery_items if it doesn't exist yet
if 'grocery_items' not in st.session_state:
    st.session_state.grocery_items = ['Apple', 'Banana', 'Carrot', 'Milk', 'Eggs']
# Streamlit app interface
st.title('Grocery List App')
# Text input to add a new item to the list
new_item = st.text_input("Add a new item to your grocery list:")
# Button to add the new item to the list
if st.button('Add Item'):
    if new_item:
        # Append the new item to the list stored in session state
        st.session_state.grocery_items.append(new_item)
        st.success(f"'{new_item}' has been added to your list!")
```

```
    else:
        st.warning("Please enter an item to add.")
# Display the current list of grocery items
st.write("### Items to Buy:")
for item in st.session_state.grocery_items:
    st.write(f"- {item}")
```

When we use **`st.session_state`**, the items we add will be preserved during the current session. On the first run, the list will contain the initial elements, and as the user adds more items, the list will grow accordingly.

However, once the page is reloaded or the session ends, the list will reset unless we store the data in a persistent location (e.g., a file or database).

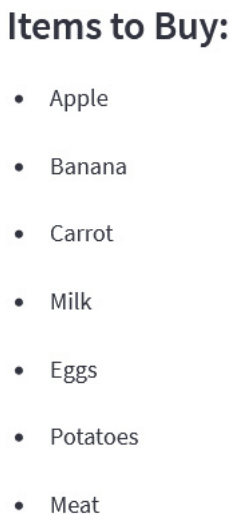


Figure 10.8 – Updated list

While using **`st.session_state`** allows temporary storage of values during a user session—gradually filling up as interactions occur—this data is lost upon a full page reload or app restart. In contrast, **`st.connection`** enables Streamlit to maintain persistent access to external resources, ensuring that data remains available across sessions and reloads. This makes it ideal for applications that require consistent interaction with long-lived data, overcoming the limitations of in-memory session state. **`st.connection`** allows the connection to external services to be maintained and reused and does so efficiently with each user interaction.

Let's see how **`st.connection`** works in practice:

```
import streamlit as st
conn = st.connection("my_database_sql")
df = conn.query("select * from my_beautiful_table")
st.dataframe(df)
```

In this section, we discussed the main components of a Streamlit application. In the next one, we will discuss how to beautify our app and make the user experience better.

Developing our frontend with Streamlit

In this section, we will begin to discuss some of the elements that allow us to improve the user experience when interacting with our app.

We will show the various frontend elements and how to combine them for complex apps.

Adding the text elements

To improve our user experience, we can start by improving the text elements. The first elements we add are the following:

- **`st.title()`**: This sets the main title of your Streamlit app. It's the largest text element and is typically used for the main heading of your app. Every app should have at least one title, and this is shown in the GitHub-flavored Markdown. This function obviously takes a string.
- **`st.header()`**: This adds a header to your app. It's smaller than the title but still stands out as an important section heading. This also has a counterpart in GitHub and is similar in purpose. One attribute you can add is **`divider`**, which shows a colored divider below the header (we can specify a color). Also, we can add a **`help`** string that provides a tooltip next to the header.
- **`st.subheader()`**: This adds a subheader, which is smaller than the header and is typically used for subsections or to provide additional structure to the content. The subheader can also have a colored divider if you want one. A help **`string`** is also possible.

Here are some examples of how to insert these elements:

```
st.title("Your Title Here")
st.header("Your Header Here")
st.header("Your Header Here", divider=True, help ="bla bla")
st.subheader("Your Subheader Here")
```

Now, we can test them directly in our app:

```
import streamlit as st
# Initialize session state for grocery_items if it doesn't exist yet
if 'grocery_items' not in st.session_state:
    st.session_state.grocery_items = ['Apple', 'Banana', 'Carrot', 'Milk', 'Eggs']
# Streamlit app interface
st.title('Grocery List App :banana: :apple: :egg:')  # Main title of the app
# Display a header for the section where the user can add items
st.header('Add new item')
# Text input to add a new item to the list
new_item = st.text_input("Type an item to add to your grocery list:")
# Button to add the new item to the list
if st.button('Add Item'):
    if new_item:
        # Append the new item to the list stored in session state
        st.session_state.grocery_items.append(new_item)
        st.success(f"'{new_item}' has been added to your list!")
    else:
        st.warning("Please enter an item to add.")
# Display a subheader for the current grocery list
st.subheader('Current Grocery List')
# Display the current list of grocery items
for item in st.session_state.grocery_items:
    st.write(f"- {item}")
```

This code shows how to start inserting stylistic elements into our app. The following figure shows the result after these improvements:

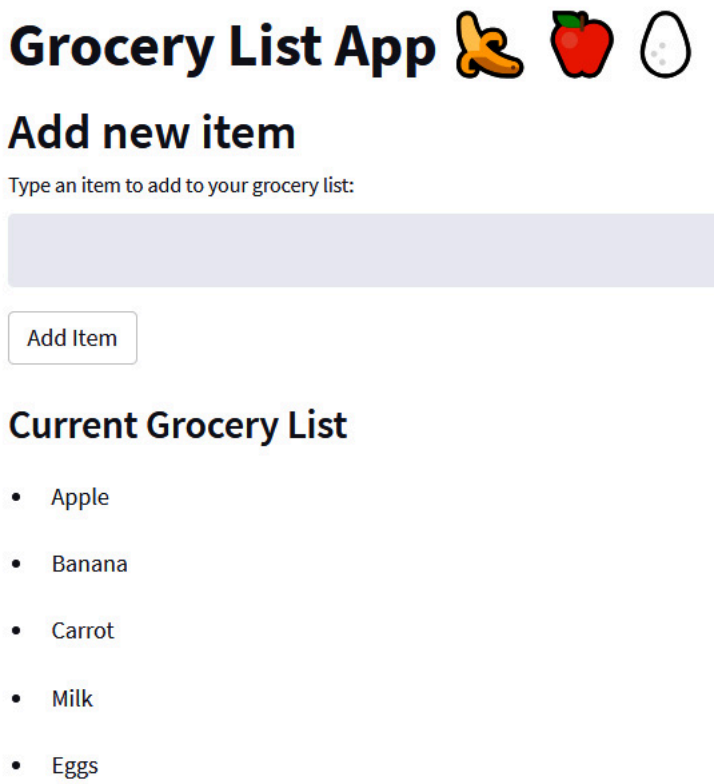


Figure 10.9 – Updated app

Inserting images in a Streamlit app

Next, we begin the customization of our app, adding both a logo and an image. To do this, we will use several elements:

- **`st.set_page_config(...)`**: This function is used to configure the Streamlit app's page settings, such as the title of the page, favicon (icon in the browser tab), and layout preferences. In this case, we will use it to add a small icon that will be seen as a browser tab element.
- **`st.image(...)`**: This function displays an image in the Streamlit app. It takes the URL or path of the image and can adjust its width to fit the screen with **`use_column_width=True`**. As input, **`st.image`** takes either a URL (as we are doing in this case) or a path to a local image or **`numpy.array`** (the image can be in number format).
  One of the keywords is **`caption`**, which allows us to provide a caption for the image directly. In our case, however, we will add the caption separately.
- **`st.caption(...)`**: This function adds a small caption or descriptive text below elements, such as images or charts. In our app, it provides the image credit.
- **`st.sidebar.image(...)`**: This places an image in the sidebar, which will be the collapsible menu on the left side of the app. The sidebar is useful for placing navigation, settings, or additional content.

We will now insert an image:

```
# Set the page configuration to include a logo
st.set_page_config(page_title="Grocery List App", page_icon="https://github.com/Sal
# Display the title image
st.image("https://github.com/SalvatoreRa/tutorial/blob/main/images/vegetables.jpg?r
```

```
st.caption("Image from [here](https://unsplash.com/it/@randyfath)")
# Add logo to the sidebar
st.sidebar.image("https://github.com/SalvatoreRa/tutorial/blob/main/images/vegetabl
```

The preceding code shows how to insert an image with the proper caption. The following figure shows the results:

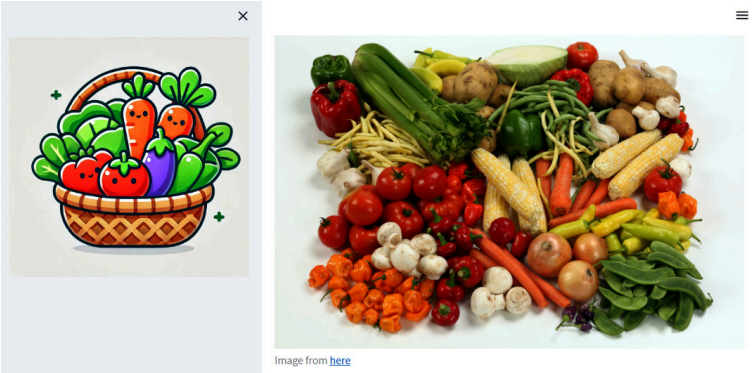


Figure 10.10 – Changes in appearance in the app

Here is our browser icon:



Figure 10.11 – The browser icon

Thus far, we have explored the basic features of Streamlit and used them to build a simple and static app. Now it's time to move beyond and start exploring what makes a Streamlit app dynamic, responsive, and connected to real use.

Creating a dynamic app

We can further modify our app to make it more dynamic. So far, our user can only add items to their list, and then the list is shown. This app is of little use, so we want to make it more dynamic and allow the user to add quantities. So, we're going to do the following:

- Allow the user to add an item to buy. Once the item is added, two sliders are created that represent the quantity the user has at home and how much they have to buy. To avoid creating an endless list, we will use two columns. In addition, we will add a button to select whether or not the user has taken the ingredient.
- Make an interactive display of a table with ingredients, showing how much to buy, how much was taken, and whether it was taken, as well as a completion bar that shows how many items have been taken and how many are missing.
- In the sidebar, add a button to download the list.

Let's start by displaying the grocery list items in a structured manner using two columns, ensuring a more compact and visually balanced layout:

```
data = []
for i, item in enumerate(st.session_state.grocery_items):
    with col1 if i % 2 == 0 else col2:
        st.markdown(f"**{item}**")
        quantity_at_home = st.slider(f"Quantity at home", 0, 12, st.session_state.q
        st.session_state.quantity_at_home[item] = quantity_at_home
```

```
            quantity_to_take = st.slider(f"Quantity to take", 0, 12, st.session_state.q
            st.session_state.quantity_to_take[item] = quantity_to_take
            taken = st.checkbox(f"Taken", st.session_state.taken[item], key=f"taken_{it
            st.session_state.taken[item] = taken
            data.append([item, quantity_at_home, quantity_to_take, "Yes" if taken else
```

For each item, we determine whether it should be placed in the first column (**`col1`**) or the second column (**`col2`**) based on whether the index, **`i`**, is even or odd. This ensures that items are distributed evenly between the two columns, preventing a long vertical list.

Inside the selected column, the item name is displayed in bold using **`st.markdown()`**. Below the name, two sliders are created: one for the quantity the user has at home and another for the quantity they need to take. Each slider is assigned a unique key based on the item name to ensure proper tracking and persistence of values. The values from these sliders are stored back into the session state so they remain updated across app interactions. In addition, a checkbox is included for each item. The collected data for each item, including its name, the selected quantities, and whether it has been taken or not, is appended to the data list.

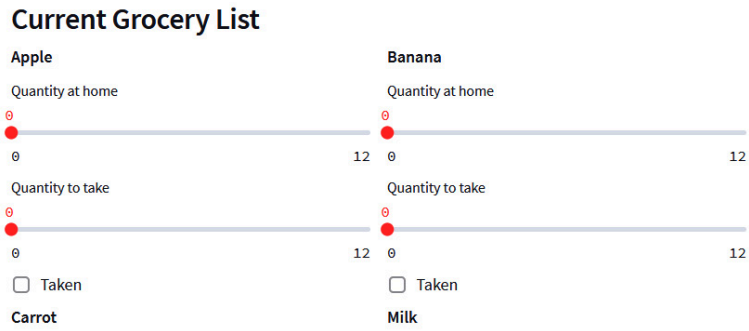


Figure 10.12 – Restyling of the app

Notice that the app is interactive (we can interact with sliders):

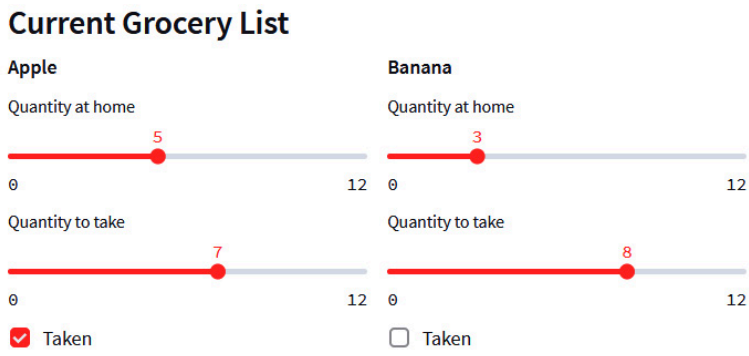


Figure 10.13 – Interactive elements

The preceding figure shows us how to insert interactive elements and how we can interact with them. Streamlit allows this in the background, without the need for us to code these complex elements, and we can use simple commands.

We can then display the table:

```
df = pd.DataFrame(data, columns=["Name", "Quantity at Home", "Quantity to Take", "T
st.table(df)
```

	Name	Quantity at Home	Quantity to Take	Taken
0	Apple	5	7	Yes
1	Banana	3	8	No
2	Carrot	0	9	No
3	Milk	0	4	No
4	Eggs	9	0	Yes
5	Potato	1	5	No

Figure 10.14 – Table obtained

At this point, we can create our progress bar:

```
# Progress bar
taken_count = sum(1 for item in st.session_state.taken.values() if item)
total_items = len(st.session_state.grocery_items)
progress = taken_count / total_items if total_items > 0 else 0
st.subheader("Grocery Completion Progress")
st.progress(progress)
st.write(f"{taken_count} out of {total_items} items taken ({progress*100:.2f}%)")
```

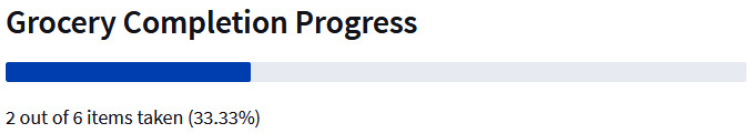


Figure 10.15 – Progress bar

Next, we define a function, **`generate_pdf()`**, which creates a PDF document containing the grocery list data and allows users to download it:

```
# Function to generate PDF
def generate_pdf():
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
    pdf.set_font("Arial", size=12)
    logo_path = "logo.jpg"  # Add logo to PDF
    response = requests.get(logo_url)
    with open(logo_path, "wb") as f:
        f.write(response.content)
    pdf.image(logo_path, 10, 10, 30)  # Position and size of the logo
    pdf.cell(200, 10, "Grocery List", ln=True, align='C')
    pdf.ln(20)  # Added extra spacing to prevent text overlapping the logo
    for index, row in df.iterrows():
        pdf.cell(0, 10, f"{row['Name']} - At Home: {row['Quantity at Home']} - To T
    pdf_output = os.path.join(os.getcwd(), "grocery_list.pdf")
    pdf.output(pdf_output)
    return pdf_output
# Directly download the PDF when the button is clicked
if st.sidebar.button("Download List as PDF"):
    pdf_file = generate_pdf()
    with open(pdf_file, "rb") as f:
        st.sidebar.download_button("Download Grocery List PDF", f, file_name="groce
```

First, we initialize an **`FPDF`** object with automatic page breaks and add a new page. The font is set to **`Arial`** with a size of **`12`** for consistent formatting. To enhance the PDF visually, the **`generate_pdf()`** function downloads a logo from a specified URL, saves it locally as **`logo.jpg`**, and embeds it in the top-left corner of the page. A centered title, **`Grocery List`**,

is added, followed by some spacing to ensure the text does not overlap with the logo. The function then iterates through the grocery list stored in **`DataFrame (df)`**, adding each item's name, quantities at home and to take, and whether the item has been marked as taken. Once the document is populated, it is saved in the current working directory as **`grocery_list.pdf`** and returned.

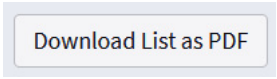


Figure 10.16 – The PDF button

Here is the generated PDF:



Figure 10.17 – The obtained PDF file

Our users may want to add notes; for this, we can take advantage of the fact that Streamlit allows other pages to be added to create a section for notes. Note that we now have a second page that we can access through our sidebar. This way, we can enter notes and then save them:

```
elif page == "Notes":
    st.title("Notes")
    st.session_state.notes = st.text_area("Write your notes here:", st.session_stat
    if st.button("Save Notes"):
        st.success("Notes saved successfully!")
```

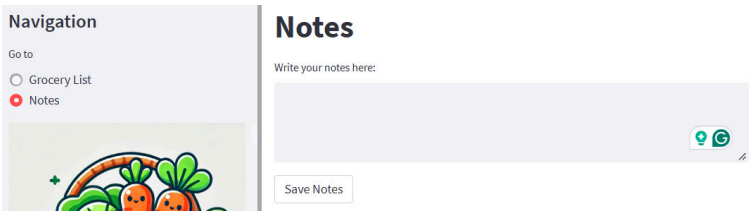


Figure 10.18 – Adding another page to the app

Now, we can also note that the information has been updated in our PDF:

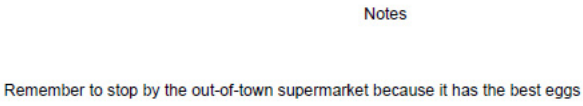


Figure 10.19 – Updated PDF

If our users want to know where the nearest supermarkets are, we could add the following functionality to our app:

```
elif page == "Find Supermarkets":
    st.title("Find Nearby Supermarkets (OSM)")
```

```
        # Get user's location
    location_input = st.text_input("Enter your location (City, Address, or Coordina
    if st.button("Find Supermarkets") and location_input:
        geolocator = Nominatim(user_agent="grocery_app")
        location = geolocator.geocode(location_input)
        if location:
            st.success(f"Location found: {location.address}")
            # Create map
            m = folium.Map(location=[location.latitude, location.longitude], zoom_s
            folium.Marker([location.latitude, location.longitude], tooltip="Your Lo
            # Use Overpass API to find nearby supermarkets
            overpass_url = "http://overpass-api.de/api/interpreter"
            overpass_query = f"""
            [out:json];
            node["shop"="supermarket"](around:5000,{location.latitude},{location.lo
            out;
            """
            response = requests.get(overpass_url, params={'data': overpass_query})
            data = response.json()
            for element in data["elements"]:
                lat, lon = element["lat"], element["lon"]
                name = element.get("tags", {}).get("name", "Unnamed Supermarket")
                folium.Marker([lat, lon], tooltip=name, icon=folium.Icon(color="gre
            folium_static(m)
        else:
            st.error("Location not found. Please try a different input.")
```

In the code, we are adding a new page to the Streamlit app where users can find nearby supermarkets using **OpenStreetMap** (**OSM**). First, we display a title for the page and add a text input field where users can enter their location, which could be a city name, an address, or coordinates. When the user clicks the **Find Supermarkets** button, we use the **`Nominatim`** geocoder from the **`geopy`** library to convert the location input into latitude and longitude coordinates. If a valid location is found, we confirm this to the user and create an interactive map centered at the given coordinates using Folium. A marker is added to indicate the user's location. Next, we use the Overpass API, which queries OSM data, to find supermarkets within a 5-kilometer radius. We send a request to the Overpass API and parse the JSON response to extract the coordinates and names of nearby supermarkets. Each supermarket is then added as a green marker on the map. Finally, we display the generated map inside the Streamlit app using **`folium_static`**. If the location input is invalid or not found, we show an error message prompting the user to try again.

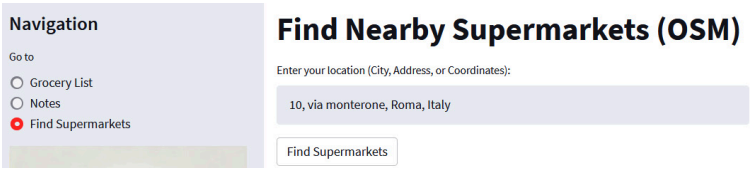


Figure 10.20 – Find Supermarkets page

When we click **Find Supermarkets**, we get the following:

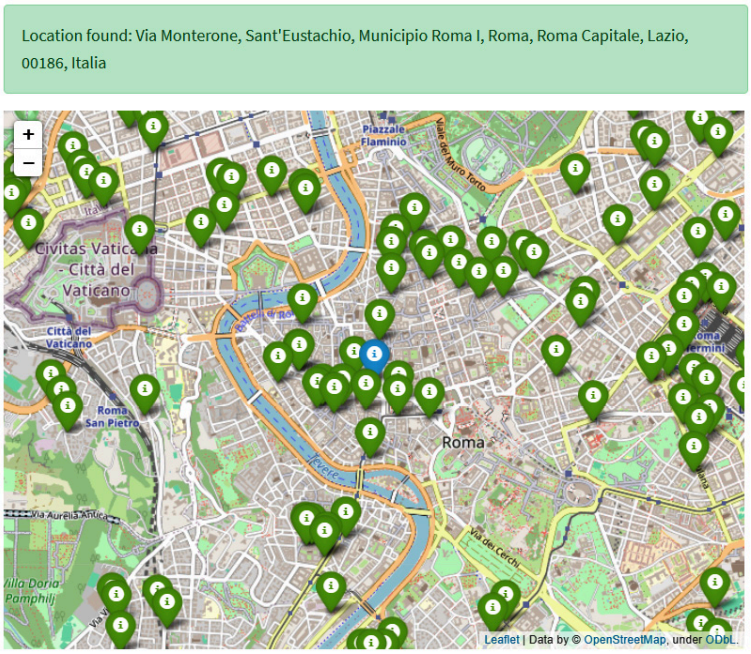


Figure 10.21 – Supermarket map

Now that we know how to build an app, we can build one with agents.

Creating an application with Streamlit and AI agents

In this section, we will look at integrating the multi-agent system described in ***Chapter 9*** into an app with Streamlit. Here, we will describe only the code parts we change; the structure remains the same. In the previous chapter, we built a script that allowed a travel program to be defined; in this chapter, the output is the same, but the system is encapsulated in an app. In other words, our app will run in the browser and can be used even by a user who does not know programming.

As a brief recap, the multi-model *Travel Planning System* is an AI-driven assistant that integrates multiple specialized models to generate personalized travel plans. It consists of four key agents:

- **`WeatherAnalysisAgent`**: Predicts the best travel months using historical weather data
- **`HotelRecommenderAgent`**: Uses a transformer model to find accommodations that match user preferences
- **`ItineraryPlannerAgent`**: Employs GPT-2 to generate detailed day-by-day travel plans
- **`SummaryAgent`**: Creates professional trip summaries and cost estimates

The system follows a structured data flow, where the user inputs their destination, preferences, and duration, and the agents collaborate to deliver a complete travel plan. The core AI models include **`RandomForestRegressor`** for weather predictions, **`SentenceTransformer`** for hotel recommendations, and GPT-2 for itinerary and summary generation.

To better understand the internal structure of the *Travel Planning System*, this section provides three UML diagrams. These visualizations illustrate

the architecture, execution flow, and system interactions of the application described in this chapter:

- **Class diagram**: The following class diagram shows the main components of the application, including the core AI agents (such as **`WeatherAnalysisAgent`** and **`ItineraryPlannerAgent`**), the underlying models (**`RandomForest`**, **`SentenceTransformer`**, and **`OpenAI`** GPT), and the Streamlit app that connects the user interface to the backend logic:

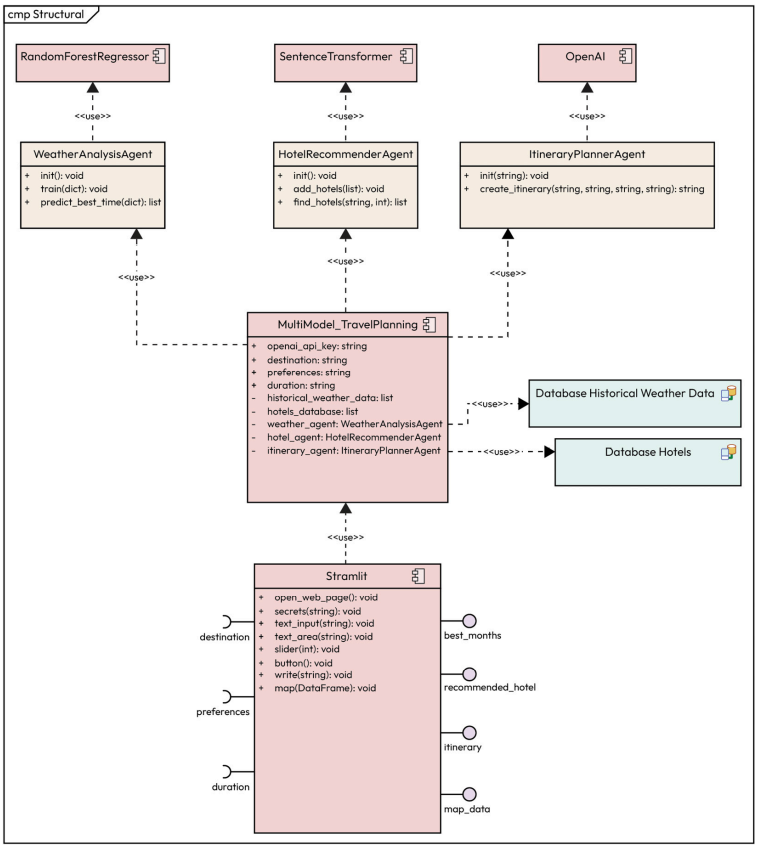


Figure 10.22 – Structural UML Diagram for the multi-model Travel Planning System

- **Activity diagram**: The activity diagram describes the control flow of the application, starting from user input collection through to the generation of a complete travel plan. It illustrates how each agent is triggered and how their outputs are merged:

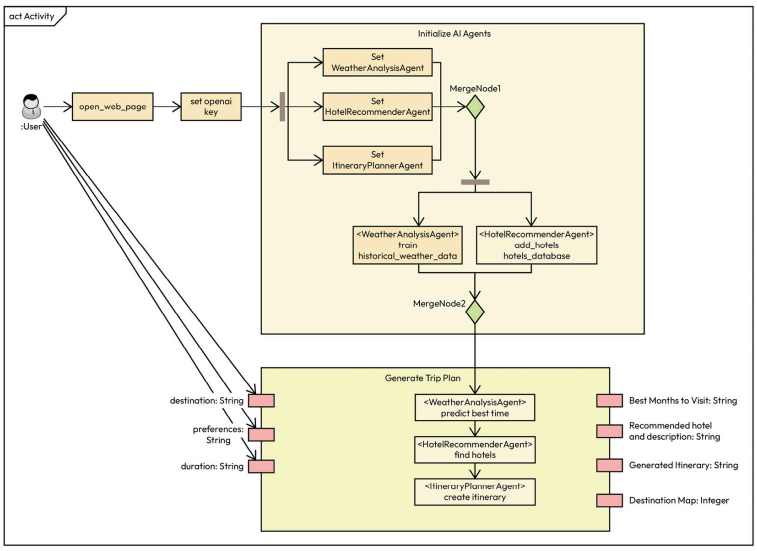


Figure 10.23 – UML activity diagram for the multi-model Travel Planning System

- **Sequence diagram**: Finally, the sequence diagram outlines the time-based interactions between the Streamlit frontend, the database, and the AI agents. It shows the order of method calls, the data exchanged, and the points where the system waits for responses. It makes clear when and how each agent is called:

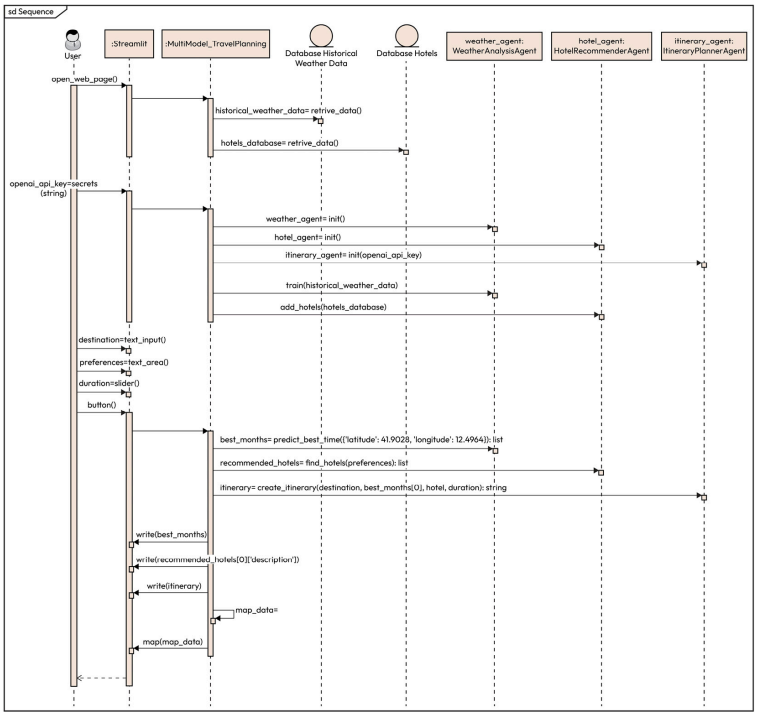


Figure 10.24 – UML sequence diagram for the multi-model Travel Planning System

First, we start by importing the libraries we need:

```
import streamlit as st
import numpy as np
import pandas as pd
import pydeck as pdk
import openai
from sklearn.ensemble import RandomForestRegressor
from sentence_transformers import SentenceTransformer
```

- **`streamlit`**: Our library to create the interactive web application
- **`numpy`**: A library for all the numerical operations
- **`pandas`**: A library to handle DataFrames
- **`pydeck`**: A visualization library built on top of Deck.gl, specifically for rendering large-scale geographical data
- **`openai`**: The OpenAI Python library, which provides access to models such as GPT-3.5 and GPT-4 for **natural language processing** (**NLP**) tasks.
- **`RandomForestRegressor`**: The scikit-learn model we use in our app
- **`SentenceTransformer`**: The library for the embeddings (see the previous chapter)

The code for agents is the same, except for **`ItineraryPlannerAgent`**. For a better and smoother response, we use OpenAI's GPT-4 model here:

```
class ItineraryPlannerAgent:
    def __init__(self, api_key):
        self.api_key = api_key
    def create_itinerary(self, destination, best_month, hotel, duration):
        client = openai.OpenAI(api_key=self.api_key)
        prompt = f"""
        Create a {duration}-day travel itinerary for {destination} in the best mont
        Recommended Hotel: {hotel['name']}.
        """
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "system", "content": "You are an expert travel planner."},
                {"role": "user", "content": prompt}
            ],
            max_tokens=300
        )
        return response.choices[0].message.content
```

The operation is the same: it takes in a travel destination, the best time to visit, a recommended hotel, and the trip duration, following which a structured itinerary is generated. Note that we need to use an API key to authenticate requests to OpenAI's API. Again, the agent does nothing more than generate an itinerary based on the same inputs: travel location, the best months to travel, hotel details, and the number of travel days. GPT-4 also works similarly to GPT-2: we have to provide a prompt with the information and the model then autoregressively generates the travel itinerary

Here again, we provide the same data that we provided to our system previously (you can find it in the repository):

```
# ------------------------------
# Sample Data
# ------------------------------
historical_weather_data = [
    {'month': i, 'latitude': 41.9028, 'longitude': 12.4964, 'weather_score': np.random.rand()} for i in
]

hotels_database = [
    {'name': 'Grand Hotel', 'description': 'Luxury hotel in city center with spa.', 'price': 300},
    {'name': 'Boutique Resort', 'description': 'Cozy boutique hotel with top amenities.', 'price': 250
    {'name': 'City View Hotel', 'description': 'Modern hotel with stunning city views.', 'price': 200}
]
```

Figure 10.25 – Screenshot of the code

At this point, we can initialize our agents, each with its own different purpose:

```
openai_api_key = st.secrets["general"]["openai_api_key"]
weather_agent = WeatherAnalysisAgent()
hotel_agent = HotelRecommenderAgent()
itinerary_agent = ItineraryPlannerAgent(api_key=openai_api_key)
weather_agent.train(historical_weather_data)
hotel_agent.add_hotels(hotels_database)
Your API should be in a file TOML, like this:
[general]
openai_api_key = "YOUR_API"
```

Note, **`openai_api_key = st.secrets["general"]["openai_api_key"]`** uses Streamlit's secrets manager to securely access the OpenAI API key. In fact, **`st.secrets`** is a way to store and retrieve sensitive credentials in Streamlit apps. The API key is stored under **`st.secrets["general"]["openai_api_key"]`**, indicating it is saved inside a

"general" section within the **secrets** configuration. The purpose of **st.secrets** is to prevent sensitive credentials from being hardcoded in the script, reducing the risk of privacy breaches.

Now, let's start building our interface:

```
st.title("AI Travel Planner ✈️")
st.write("Find the best time to travel and discover the perfect hotel!")
destination = st.text_input("Enter your destination (e.g., Rome):", "Rome")
preferences = st.text_area("Describe your ideal hotel:", "Luxury hotel in city cent
duration = st.slider("Trip duration (days):", 1, 14, 5)
```

First, we add a title: **st.title()** sets the title of the Streamlit web app. This title will appear at the top of the page. At this point, we use **st.write()** to give a brief explanation of the app's purpose. Next, **st.text_input()** is used to create a box where the user can enter their destination. Note that we are providing a hint about what the user can enter – **"Enter your destination (e.g., Rome):"** – and there is a default value of **"Rome"** (if the user doesn't input anything, it defaults to **Rome**). **st.text_area()** creates a multi-line text box where users can describe their ideal hotel. We use **text_area** to allow users to provide detailed hotel preferences. **st.slider()** creates a slider input for selecting the trip duration (there are parameters that define a minimum duration of **1** day and a maximum of **14**, with a **5**-day trip being the default duration).

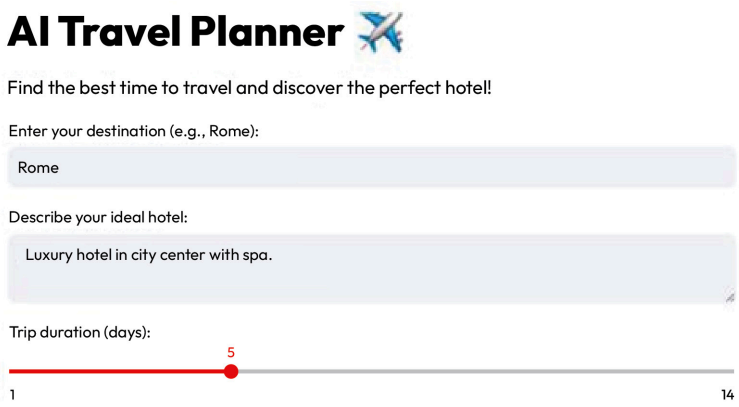


Figure 10.26 – Input preferences in the app

At this point, we will deal with what happens after the user adds the information and presses a button. To recap, the system predicts the best travel months based on weather conditions (through the use of historical data and random forest algorithms), finds a hotel that matches the user's preferences (using data on hotels and similarity of embeddings), and finally, creates a personalized itinerary using OpenAI's GPT-4.

We have created the framework to be able to visualize the results: the best months to visit, the recommended hotel, the AI-generated itinerary, and finally, a map visualization of the destination. All this happens only when our user presses the button, which we will create next:

```
if st.button("Generate Travel Plan ✨"):
    best_months = weather_agent.predict_best_time({'latitude': 41.9028, 'longitude'
    best_month = best_months[0]['month']
    recommended_hotels = hotel_agent.find_hotels(preferences)
    itinerary = itinerary_agent.create_itinerary(destination, best_month, recommend
    st.subheader("📆 Best Months to Visit")
```

```
    for m in best_months:
        st.write(f"Month {m['month']}: Score {m['score']:.2f}")
    st.subheader("🏨 Recommended Hotel")
    st.write(f"**{recommended_hotels[0]['name']}** - {recommended_hotels[0]['descri
    st.subheader("📜 Generated Itinerary")
    st.write(itinerary)
    # ------------------------------
    # Interactive Map
    # ------------------------------
    st.subheader("🗺️ Destination Map")
    map_data = pd.DataFrame(
        {'lat': [41.9028], 'lon': [12.4964]},
    )
    st.map(map_data)
```

`if st.button("Generate Travel Plan ✨"):` creates an interactive button labeled **Generate Travel Plan ✨** and defines a block of actions that are executed when the user clicks the button. First, the best month to visit the city is predicted: **`best_months = weather_agent.predict_best_time({'latitude': 41.9028, 'longitude': 12.4964})`**. Note that we entered the destination's latitude (**`41.9028`**) and longitude (**`12.4964`**) for Rome, got our best months based on the weather score, and selected the best month. At this point, we identify the best hotels based on our user's preferences with **`hotel_agent.find_hotels(preferences)`**. This agent will return a list of hotels matching the user's description.

Since we have all the details, we can generate our itinerary. **`itinerary = itinerary_agent.create_itinerary(destination, best_month, recommended_hotels[0], duration)`** does exactly that; it takes the inputs defined earlier and produces a structured AI-generated itinerary. Once we have our itinerary, we start the display of it for the user. We use **`st.subheader("📆 Best Months to Visit")`** to create a subsection and then iterate over **`best_months`** and print each month with its weather score. At this point, we show the best hotels in an additional subsection after **`st.subheader("🏨 Recommended Hotel")`**. Finally, **`st.subheader("📜 Generated Itinerary")`** allows us to create a subsection where our itinerary will be inserted. In the last part, we show the city map.

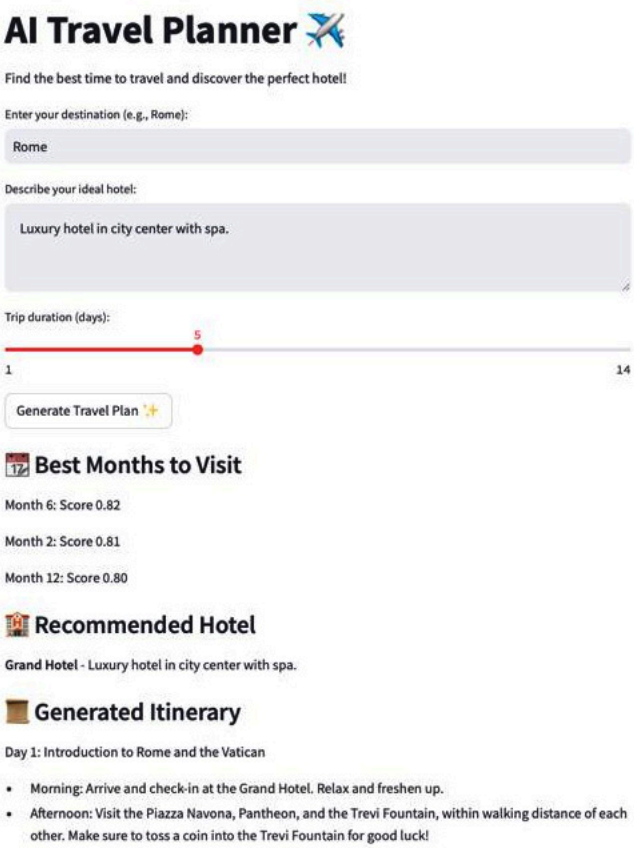


Figure 10.27 – Generated output (part 1)

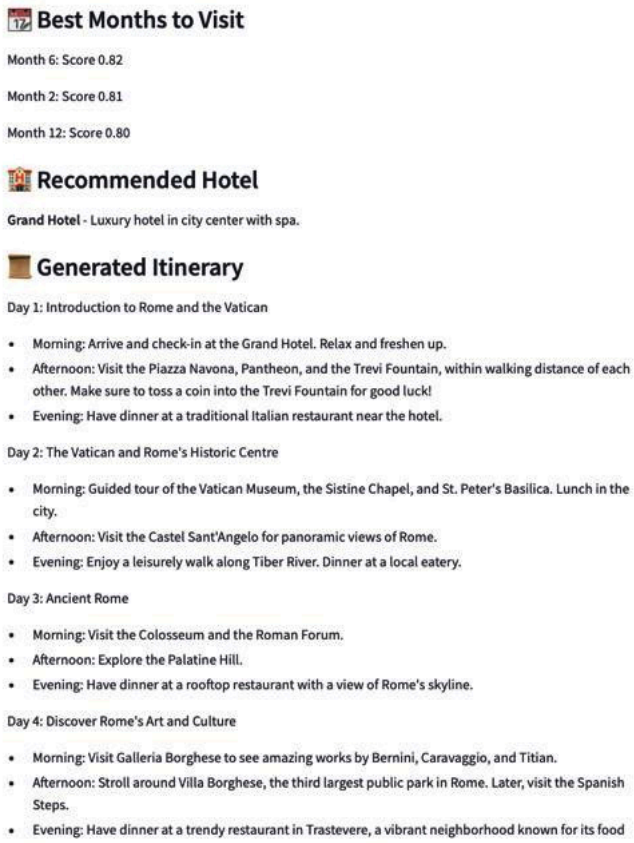


Figure 10.28 – Generated output (part 2)

- Afternoon: Visit the Castel Sant'Angelo for panoramic views of Rome.
- Evening: Enjoy a leisurely walk along Tiber River. Dinner at a local eatery.

Day 3: Ancient Rome

- Morning: Visit the Colosseum and the Roman Forum.
- Afternoon: Explore the Palatine Hill.
- Evening: Have dinner at a rooftop restaurant with a view of Rome's skyline.

Day 4: Discover Rome's Art and Culture

- Morning: Visit Galleria Borghese to see amazing works by Bernini, Caravaggio, and Titian.
- Afternoon: Stroll around Villa Borghese, the third largest public park in Rome. Later, visit the Spanish Steps.
- Evening: Have dinner at a trendy restaurant in Trastevere, a vibrant neighborhood known for its food

Destination Map



Figure 10.29 – Generated output (part 3)

In this section, we created a multi-agent system and embedded it within an app. In this way, even users with no programming knowledge can interact with our system. The system can be run by a user by clicking a simple button.

We discussed an app as an isolated system; in the next section, we will see how a model is not an isolated concept but part of an ecosystem. This complexity must be taken into account, and in the next section, we will discuss the life cycle of a model, from conception to deployment.

Machine learning operations and LLM operations

We have seen how to create an app containing a multi-agent system. When we create a script with Python, we create an element that can run on our computer, but this is not a product. Turning a script into an app allows a user to be able to interact with our app even if they do not know how to program. Streamlit allows us to be able to run a quick prototype of our app. This is not optimal for a product, especially if it is to be used by several users. In this section, we will discuss all those operations necessary to make our model function as a product.

Machine Learning Operations (MLOps) is a set of practices and tools designed to streamline and manage the life cycle of **machine learning (ML)** models in production. It combines ML, DevOps, and data engineering practices to ensure the **continuous integration/continuous delivery (CI/CD)**, monitoring, and scaling of ML systems.

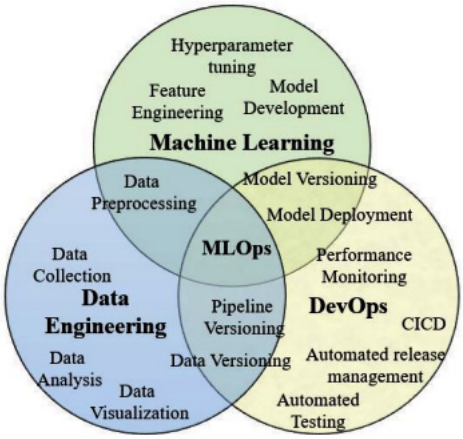


Figure 10.30 – MLOps combination (https://arxiv.org/pdf/2202.10169)

MLOps plays a key role in turning a model into a useful application in the real world. In short, MLOps encompass the development, monitoring, and maintenance of models in a production environment, enabling the transition from a research product to a functional product. Here are the various stages involved:

1. **Model development**: This is the first step, in which an ML model is designed and trained. Typically, at this stage, both data scientists and data engineers collaborate on the choice of model, datasets, and training and testing process.
2. **Testing**: Normally, the testing phase is part of model development; however, today, there is a greater emphasis on testing the model. Hence, we consider it a separate stage. In fact, complex models in particular can exhibit unexpected behaviors, so testing is often considered a separate phase.
3. **Deployment**: Once the model has been developed and tested, it can be deployed in a production environment. This delicate step requires that the model be integrated with other existing systems (which have been developed previously) and that it can be used in real time.
4. **Monitoring and maintenance**: Once the model is deployed, we must ensure its performance doesn't degrade and prevent operational problems. At the same time, we may need to update the model or ensure compatibility with new system elements.

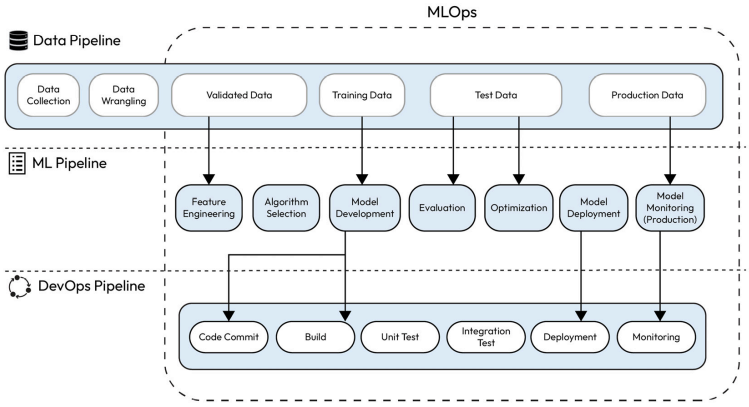


Figure 10.31 – High-level process view of MLOps (https://arxiv.org/pdf/2202.10169)

Large Language Model Operations (LLMOps) is an extension of MLOps specifically focused on the deployment, maintenance, and management of LLMs. It incorporates the principles of MLOps but also addresses the

unique challenges and needs associated with working with large-scale NLP models.

However, LLMOps adds additional complexity. Here's why:

- **Model size and complexity**: In MLOps, models can vary in size and complexity, but they typically don't require as much computational power or memory as LLMs. Models may include traditional ML algorithms, smaller deep learning models, or specialized models for structured data. LLMs can be in the order of billions of parameters and thus require optimized infrastructure (often involving specialized hardware such as GPUs or TPUs) or distributed training. This means more expertise and dedicated infrastructure (dedicated hardware and storage), which can be very expensive.
- **Training and fine-tuning**: In MLOps, training is much more manageable. Many of the models are small in size and can therefore be easily retrained. Retraining itself can be conducted programmatically. Fine-tuning LLMs is more complex and resource-intensive. Collecting and processing the datasets needed for an LLM is resource-intensive.
- **Scalability and deployment**: In MLOps, deploying models to production is usually straightforward. Scaling LLMs, on the other hand, requires dedicated infrastructure that can ensure necessary support when there is high demand. In fact, latency can increase considerably when there are many users. Optimizing latency during inference can be a delicate process that risks degrading performance.
- **Monitoring and maintenance**: Monitoring ML models in production involves tracking key metrics such as accuracy, precision, and recall, as well as model drift or data drift. Monitoring LLMs involves not only the usual performance metrics but also the quality of text generation, user feedback, and ethical concerns such as biased or harmful outputs. While it is straightforward to evaluate an output in terms of accuracy, it is more complex to assess whether an LLM produces hallucinations or inappropriate or harmful content. Some biases might be subtle but still be noticed by users.
- **Model governance and compliance**: While governance and compliance are critical in any ML deployment, MLOps primarily focuses on ensuring data privacy and model transparency, especially when dealing with sensitive or regulated data. For LLMOps, there is not only privacy, but it can also be used to generate text on a wide variety of topics with the risk of generating inappropriate content. With regulations in development, assessing bias, fairness, and ethical issues is complex and evolving.

Here's an example of the added complexity involved when performing LLMOps. If we wanted to train a model from scratch, we would have to retrieve a corpus of at least 1B tokens. These tokens would have to be collected from different sources (books, websites, articles, code repositories, and so on). In MLOPs, we usually create a model when a dataset is already present (e.g., through user interactions with our site). The steps of preprocessing a dataset for a classical model (images or tabular) are much simpler than a large corpus (steps such as debiasing, eliminating duplicates, and so on). Also, since our dataset can be over hundreds of terabytes in size, there is more complexity. While we can train an ML model easily (even on a consumer computer), this is no longer possible with an LLM. Especially for larger ones, we have to use dedicated infrastructure, and we cannot do many experiments (testing different hy-

perparameters or different architecture combinations). Similarly, fine-tuning will be preferred to having to retrain our model.

Testing also no longer relies on simple measures (such as accuracy) but requires human-in-the-loop evaluations. Given the language-centric nature of the system, a metric such as accuracy gives us only partial information about the output of our model. Only humans (even if we use other LLMs to check at scale) can evaluate the output of an LLM in terms of creativity, bias, quality, and the presence of inappropriate content. Also, after pre-training, there is usually a step where human feedback is used to be able to further improve the output of a model. In addition, we must then continue to evaluate our LLM, because the traffic may grow or there may be evolutions in the language and knowledge that our model must have. For example, an LLM for medical use needs to be updated on new therapies.

In the next section, we will start with the complexities of developing a model as complex as an LLM.

Model development

The development of a model starts with the collection of a corpus. This collection is generally divided into two types: general data and specialized data. General data represents data such as web pages, books, and conversational text. Specialized data, on the other hand, is data that is designed for a specific task, such as multilingual data, scientific data, and code:

- **General data**: Considering the large amount of data on the internet, it is now common for data collection to start with using datasets of downloaded pages or even conducting crawling to collect new data. In addition, there are also datasets of conversations (such as discussions on Reddit or other platforms), chats with LLMs, and other sources. Books are another popular source for training, as they generally contain coherent, quality text on disparate topics. These datasets contain a mixture of quality data (such as Wikipedia and blog posts) but also a large amount of data that needs to be removed, such as spam, toxic posts, and so on.
- **Specialized text data**: Today, it is common to add a multilingual corpus to improve the language capabilities of LLMs (e.g., PaLM covers 122 languages due to the addition of a multilingual corpus). Adding scientific text enables improved performance in scientific and reasoning tasks. Huge datasets of articles exist today that are ready to use and can be directly added. Almost all modern pre-training datasets also insert code. The addition of code and other structured data appears to be related to an increase in performance in some reasoning tasks.

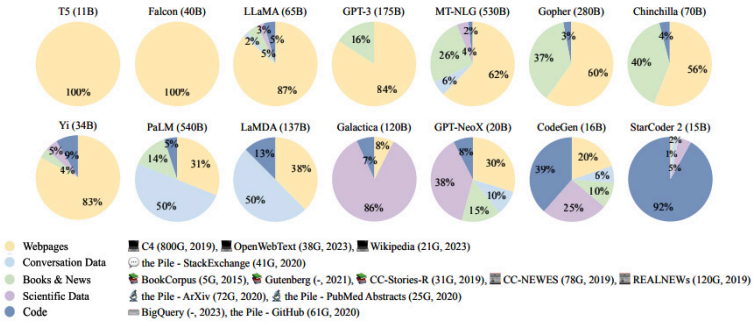


Figure 10.32 – Ratios of various data sources in the pre-training data for existing LLMs (https://arxiv.org/pdf/2303.18223)

Once the data has been collected, it must be preprocessed to remove unnecessary tokens such as HTML tags or other presentation elements, reduce text variation, and eliminate duplicate data. Today, we try to eliminate data that is of low quality, using either heuristic algorithms or classifiers. For example, we can train a classifier on quality data such as Wikipedia to recognize what content we want to preserve. Heuristic algorithms, on the other hand, rely on a set of rules that are defined upstream (such as statistical properties, the presence or absence of keywords, and so on). Deduplication is an important step because it impacts model diversity and training stability. Typically, different granularities, such as sentence or document level, are used to avoid repetitive word patterns. In addition, another common step today is privacy reduction, in which an attempt is made to remove **personally identifiable information** (**PII**), often through a set of rules that are defined upstream. Once these steps are conducted, tokenization can be done. Tokenization is considered a crucial step because it largely impacts model performance. **Byte-pair encoding** (**BPE**) tokenization is generally one of the most widely used methods.

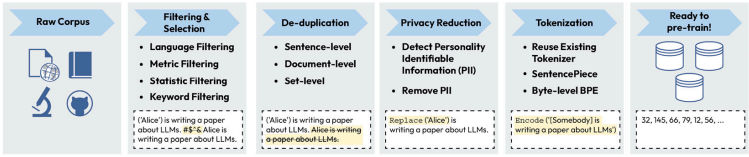


Figure 10.33 – Illustration of a typical data preprocessing pipeline for pre-training LLMs (https://arxiv.org/pdf/2303.18223)

Once we have preprocessed the corpus, we can train the model in the next phase. To train the model, we need to define a strategy to schedule the multi-sources (different types of data such as Wikipedia, text from the internet, books, etc.) previously introduced. In fact, two important aspects are decided: the proportion of each data source (data mixture) and the order in which each data source is scheduled for training (data curriculum). Since each type of data has an impact on performance, the data must be mixed in a precise distribution. This distribution can be global or local (at certain training steps). To do this, we can then decide to conduct upsampling and downsampling of the various sources in order to respect the mixture we have decided on. For example, in the case of LLaMA pre-training, the authors chose to train with the following proportion (based on experimental results, which have shown that this proportion works well): 80% web pages, 6.5% code-related data from GitHub and Stack Exchange, 4.5% from books, and 2.5% of scientific data sourced from arXiv. These values do not sum to exactly 100%, as the remaining portion includes other minor sources not explicitly detailed in the original paper. Today, this recipe has been used for many different types of LLMs, while

LLMs with a specific purpose have a different proportion of code and scientific articles.

Generally, a heterogeneous corpus is preferred, as diversity enhances a model's ability to generalize across domains. In contrast, an overly homogeneous dataset can hinder generalization. Additionally, the sequence in which data is presented—often referred to as a data curriculum—is crucial. The training data is thus typically organized to first develop foundational skills, followed by more specialized capabilities.

To do this, you first use easy/general examples and then add examples that are more complex or more specific. For example, for models that are code-specific such as **`CodeLLaMA-Python`**, the order is as follows: 2T general tokens, 500B code-heavy tokens, and 100B Python-heavy tokens.

In general, it is important that we create pipelines that allow us to collect and organize data. Generally, these kinds of pipelines are called **extract, transform, load (ETL)** pipelines. So, if we want to download a set of web pages, we will need to create an ETL pipeline that allows us to download the pages and load them into a database along with a set of metadata. The metadata will then be used both to clean the data and for data scheduling. Once the data is downloaded it needs to be transformed. Because our corpus contains different types of data, it is good to have different pipelines for preprocessing the different types (removing HTML tags from web pages, removing comments from code, and so on).

In addition, data is an important resource, and access must be controlled. Indeed, we need to prevent data leakage and ensure that our corpus complies with regulations such as the **General Data Protection Regulation (GDPR)**. Often, **role-based access control (RBAC)** is also implemented, where different users have control over a different corpus of data. For example, administrators or analysts may have different privileges so as to avoid contamination or problems with the data.

Once we have our data and have cleaned it, we create features (i.e., the data that will be used for training). The feature store is typically a database that is optimized to enable training. The idea is to have a dedicated database that we can efficiently use for training.

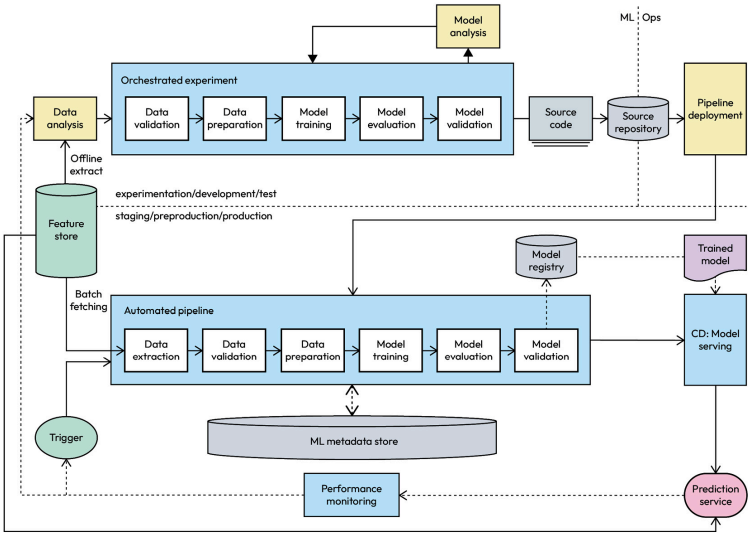


Figure 10.34 – Automation of the ML pipeline for continuous training (https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning)

Model training

Once we have our features, we need to decide what our foundation model will be. There are two alternatives: use an LLM that has already been trained or conduct fine-tuning of an already trained model. In the first case, most models today are causal decoders (as we saw in *Chapters 2* and *3*). Although the structure remains the base, there are now different alternatives and modifications (such as the mixture of experts architecture) and modifications to the attention mechanism to increase context and reduce computational cost. Training an LLM from scratch is very expensive, however, so most companies focus on using a pre-trained model and conducting fine-tuning.

Therefore, choosing the foundation model will be an important task. First, we must choose a model that has the desired performance in terms of output quality. Obviously, the chosen model must be compatible with the resources available to us (hardware and cost). In addition, we may want to choose a model that exhibits lower performance on general benchmarks but superior performance on some other aspects. For example, if our application focuses on having a coding assistant, it is better to have an LLM with superior performance on coding benchmarks than an LLM that has better wide-ranging capabilities.

When choosing a model, we need to take into account that its size impacts both its memory footprint and its storage. A larger size means higher costs in general, especially if we use a cloud provider. Also, not all models can be used for all applications (for example, we cannot use large models for specific devices). In addition, a larger model also has higher latency (the time to process an input and produce an output). A high latency disrupts the user experience and may lead the user to choose a competitor. As we saw in ***Chapter 3***, techniques (distillation, quantization, and pruning) to reduce model size while maintaining performance exist today. Another important point is the licensing of the model. Not all models have an open source license; some models may be available in repositories but may not be commercially usable.

Fine-tuning is intended to enable the model to acquire specific skills or some particular knowledge. In the former case, it is often referred to as instruction tuning. Instruction tuning is a subcategory of the supervised training process that aims to make the model more capable of following instructions or being trained for specific tasks. In repositories, there are often models that have been simply pre-trained or ones that have already undergone an instruction-tuning step. If we want the model to acquire a specific set of skills, it might be more interesting for us to collect a dataset for instruction tuning. Again, some caveats apply:

- **Data distribution**: Instruction tuning considers a mix of different tasks, so our dataset should respect this principle and contain several examples. Ideally, these examples should be of different topics, different contexts, different lengths, different styles, and different types of tasks.
- **Dataset quality**: Generally, in this step (quality check), it is important to use examples that are correct not only in terms of factual correctness but also in terms of ensuring that the task is done correctly and is well explained. For example, chain-of-thought examples are used today, where the intermediate thinking is explained instead of just the

solution. The examples are human-generated; however, to save costs, a larger model can be used initially to create the dataset for instruction tuning. For instance, a 70-billion-parameter model could be used to prepare the dataset for tuning a 7-billion-parameter model.

- **Complexity**: In general, we want our model to acquire capabilities. Through simple examples, the model will learn structure and gain a general understanding of the task. However, there should also be examples in the dataset that are difficult, require multi-step reasoning, or are complex in nature. These examples reflect the complexity of real-world problems and have been seen to help the model improve its reasoning skills.
- **Quantity**: There is also a discourse associated with quantity. According to some studies, larger models need fewer examples. For example, models with 70 billion parameters might require as few as 1,000 quality examples. In contrast, smaller models might need many more examples. Smaller models may need many examples just to understand the task and many more to master it. A 7 billion model may use up to a million examples.

Building a dataset of thousands of examples can be particularly expensive. In many studies, only a small portion is created by humans. To reach the desired number of examples, one can either use a model to generate them or integrate already available datasets. Hugging Face contains many datasets for instruction tuning, for both general purposes as well as specific domains.

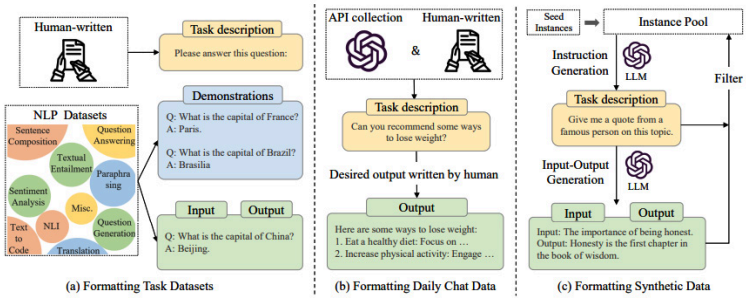


Figure 10.35 – Constructing an instruction-tuning dataset (https://arxiv.org/pdf/2303.18223)

The construction of these datasets, especially for particular domains, also requires the presence of experts (for example, if the dataset is for finance or medicine, collaboration with experts in the field or other institutions is common). Similar to a pre-training dataset, this dataset will undergo preprocessing. For example, examples of poor quality will be filtered out (one of the most commonly used methods is to have a list of keywords that indicate inappropriate content, off-topic examples, and so on), and filters will be used for length (e.g., examples that are too short or too long for the model) and for format (for some tasks, examples are formatted in a particular way, and examples that do not comply are removed). This dataset will also be deduplicated, and examples that are too similar are also often removed (if you ask an LLM to generate the examples, it might happen that examples that are too similar are generated). Patterns such as embeddings can be used for this task, where examples that have too high a similarity are filtered out. **MinHash** is another popular alternative to reduce the computational cost of the task. MinHash generates compact representations of patterns (of vectors), which are then compared with a similarity function.

Because we are interested in model performance for specific tasks, an additional step is also conducted: **data decontamination**. This is a process in which we ensure that our instruction-tuning dataset does not contain examples that are the same or too similar to those in the evaluation or test set. In fact, once we have instruction-tuned our model, we want to test it on test sets that we set aside. If there were examples that were too similar, we could not verify overfitting or storage phenomena. Data decontamination is conducted with techniques similar to data deduplication.

Before proceeding to the actual training, an additional step, **data quality evaluation**, is usually conducted. The dataset is evaluated for several criteria such as quality, accuracy, and complexity. Usually, some statistical parameters (such as the loss) are calculated and some examples are manually inspected. Recently, it has become increasingly popular to use **LLM-as-a-judge**, a strategy in which an LLM evaluates the quality of some examples. In such cases, an LLM is given a kind of template to check the quality of the examples by providing a score. Alternatively, today, there are also specific templates trained to provide a quality score. For example, reward models such as **ArmoRM-Llama3-8B-v0.1** are trained to produce an output that represents the quality of a text in terms of helpfulness, correctness, coherence, complexity, and verbosity.

Model testing

Once we have our dataset, we can conduct fine-tuning. Fine-tuning allows us to steer the capabilities and knowledge of our LLM. We must keep in mind that fine-tuning is not a magic potion; it has both risks and benefits. For example, fine-tuning exploits pre-existing knowledge of the model, but also conducts a refocus for a specific domain. This can lead to performance degradation and hallucinations. For this reason, in *Chapters 5–7*, we looked at alternatives (RAG and GraphRAG). In ***Chapter 3***, we saw that there are now also efficient fine-tuning techniques such as LoRA and QLoRA that make the process much less expensive. Today, different libraries can conduct fine-tuning of these models, such as TRL (a library created by Hugging Face), Unsloth, and Axolotl based on Unsloth; these libraries also have additional features.

After training, the key step is LLM evaluation. In general, evaluation is carried out in three stages:

- **During pre-training**: During this step, the training of the model is monitored, and, in general, metrics such as training loss (a metric based on cross-entropy), loss on the validation set, perplexity (the exponential of training loss, one of the most commonly used metrics), and gradient norm (which indicates whether there were any instabilities in the training) are evaluated.
- **After pre-training**: Once pre-training is completed, a capability analysis is conducted on the benchmark datasets. In these datasets, both model knowledge and the ability to solve certain problems are evaluated. For example, MMLU tests model knowledge on a large number of domains, while datasets such as HellaSwag test the model on reasoning skills.
- **After fine-tuning**: After instruction tuning, qualities such as the LLM's ability to follow instructions, converse, and use tools, for example, are usually evaluated. Since fine-tuning allows you to adapt the model to a

specialized domain, it is beneficial to use specialized benchmarks in such cases. For example, for medical knowledge, a dataset such as Open Medical-LLM Leaderboard can be used, or for coding skills, BigCodeBench Leaderboard is a popular choice.

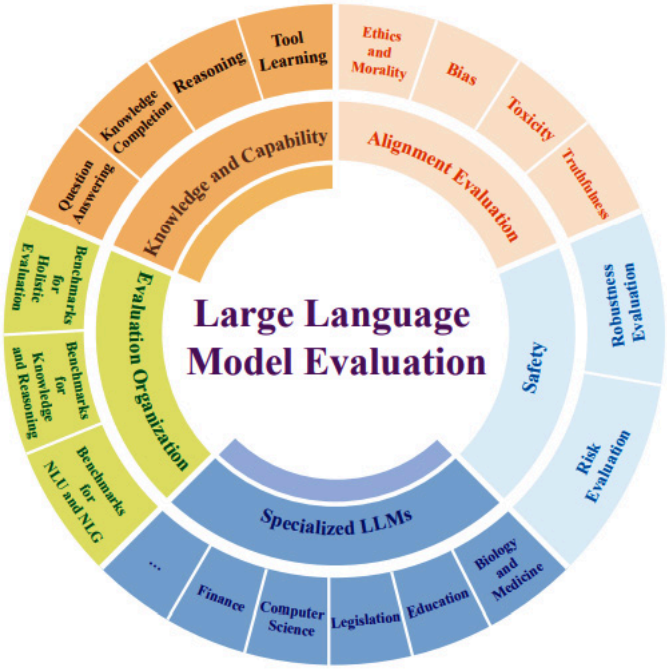


Figure 10.36 – Taxonomy of LLM evaluation (https://arxiv.org/pdf/2310.19736)

The last two steps (*After pre-training* and *After fine-tuning*) can also be conducted by manual inspection or using LLM-as-a-judge. For example, for open-ended text generation, it is more difficult to evaluate the capabilities of a model with standard metrics. Moreover, evaluating a model's capabilities in a specific domain requires more in-depth analysis.

If our LLM is a component of a system such as RAG, not only should the capabilities of the LLM be evaluated but the whole system as well. Indeed, we can evaluate the reasoning or hallucination capabilities of a model alone, but since the model will then be part of a system, we need to evaluate the whole product. For example, we should evaluate the whole RAG system for accuracy in retrieval and response generation. Even for RAG, there are both metrics and specific libraries for evaluating the system. For example, RAGAS (Retrieval-Augmented Generation Assessment) uses an LLM to evaluate the RAG response. ARES (Automatic RAG Evaluation through Synthetic data) is a comprehensive tool that takes advantage of synthetic data generation to assess model quality.

Inference optimization

Our LLM has to be deployed and will consume resources; our goal now is to optimize the inference process to avoid users encountering latency and reduce costs for us. Basically, three processes occur in inference:

1. **Tokenization and embedding**: Input is transformed into a numerical representation and then vector.
2. **Computation**: A key and value are computed for each multi-head attention.
3. **Generation**: Output is produced sequentially.

The first two steps are expensive but are easily parallelized on GPUs. The third step, on the other hand, is sequential because each output token depends on the previous token. The purpose of inference optimization is to speed up these three steps, and in this subsection, we will look at some techniques.

Model inference optimization

To produce a token output, we need all the previous context. For example, for the 15th token produced, we should calculate the **key-value** (**KV**) product of all tokens, 1 through 14. This makes the process very slow, reducing over time such that the attention has a quadratic cost ($O(n^2)$). The KV cache caches and reuses the key (*K*) and value (*V*) tensors from previous tokens, allowing faster computation of attention scores. This reduces memory and computational cost, enabling near-linear time ($O(n)$) inference. Typically, the process works like this: for the first token, we compute and store *(K,V)*. For the second, we find *(K,V)* again and add *K,V*. In other words, attention is applied only to the new tokens. As we saw in ***Chapter 2***, this is the calculation of attention:

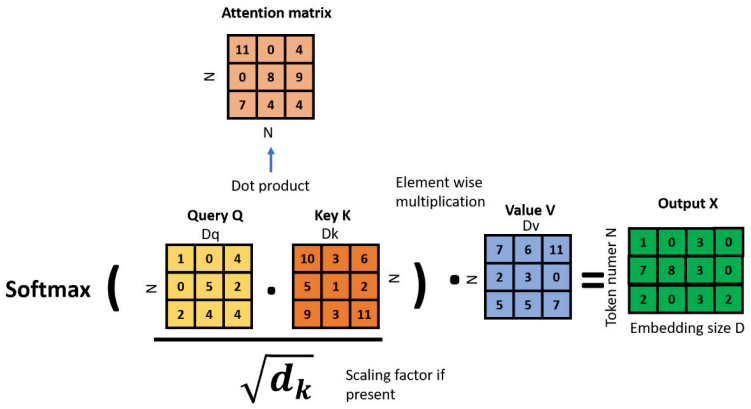


Figure 10.37 – Attention calculation

In the KV cache, we calculate the KV product, and then we save the product result in memory. At the time of a new token, we retrieve this information (the KV product) and calculate the KV product only for that token.

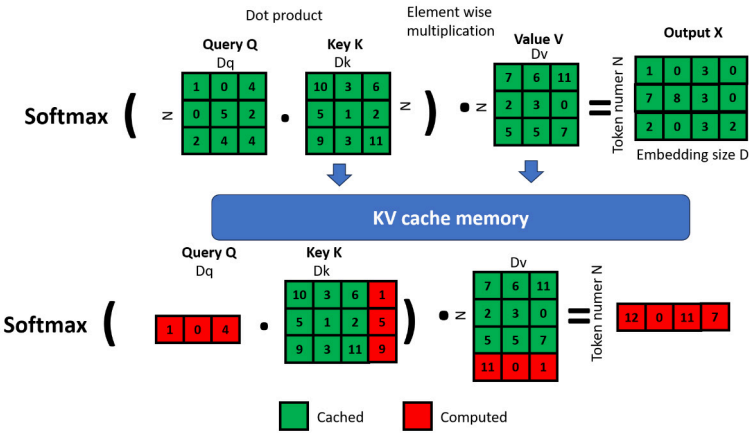


Figure 10.38 – KV cache process

The KV cache speeds up inference by eliminating some redundant computation (it prevents us from reprocessing all the previous parts of the sequence), scales well with long context windows, and is now optimized for major libraries and hardware. Of course, using the KV cache means we use more memory. In fact, it means that we have to keep in memory each KV cache per token, per attention head, and per layer. This, in practice,

also places a limit on the size of the context window we can use. Obviously, during model training, it is of little use because we have to conduct parameter updates. Therefore, today, there are approaches that try to compress the KV cache so as to reduce the cost in terms of memory.

Another technique used to speed up inference is **continuous batching**. The main purpose of this technique is to parallelize the various queries, then divide the model memory cost by the batch and transfer more data to the GPU. Traditional batching leads to slower input processing and is not optimized for inference, where the various queries may differ in size. Continuous batching, on the other hand, allows multiple user requests to be handled dynamically, allowing multiple inference requests to be processed in parallel, even if they arrive at different times. Requests that arrive at a different time are dynamically grouped into a series of batches, instead of having a fixed batch to fill. A batching engine merges multiple users' prompts into a single batch. Instead of waiting for an entire batch, new tokens are processed when resources are available. This technique also works well with the KV cache; some tokens may have already been processed and we can recall what is in memory to further speed up the process. Continuous batching thus allows lower latency, allows streaming for several users at the same time, and improves resource utilization. Of course, it is more complex than the standard implementation of attention and requires a different implementation: we have to manage users optimally, and numerous requests are made to the KV cache.

Speculative decoding is another optimization technique used in autoregressive language models to accelerate text generation. Classic LLMs generate only one token at a time, and token generation is not parallelizable, leading to inefficient inference. In speculative decoding, we have two models working together:

- A small, faster "draft" model that generates multiple candidate tokens
- The main, larger LLM that verifies the candidates and either accepts or corrects them

The draft model (a small model of the same LLM architecture as the main one, but with fewer parameters) generates multiple speculative tokens at once. The main LLM checks these proposed tokens; if they match those of the larger LLM's output, they are accepted. If, however, there is no match, the LLM discards them and continues to generate. The process is iterative until the output is finished. Speculative decoding makes it possible to reduce the number of sequential steps in inference, speed up the response, and maximize GPU consumption without losing quality. Of course, the draft model must generate good candidates; if the small model is not accurate, we lose the advantage in speedup, which means we would require another model. This approach works better with long-form than small outputs.

Another way to speed up inference is to use specific forms of attention. **Paged attention** is an optimized memory management technique for handling large KV caches efficiently during LLM inference. It works like a virtual memory system by dynamically managing memory allocation and preventing fragmentation. It is inspired by the management of memory systems in computers, and instead of storing KV caches in a continuous memory block (which can lead to fragmentation), it stores them in smaller memory pages. This allows faster retrieval of information (and only

necessary information) from the KV cache. Paged attention thus prevents GPU memory fragmentation, makes the system more efficient for long context (reduces memory consumption for long chats between the user and the system), and decreases latency by allowing easier fetching from the KV cache. **FlashAttention** is another way to make the inference process more efficient, allowing faster processing of attention with decreased memory consumption. It achieves this by processing attention in small blocks instead of storing large intermediate matrices. In this way, it makes more efficient use of GPU resources. In FlashAttention, only small blocks of various tokens are stored in the RAM. Today, many models use forms of attention during training that are aimed at faster reasoning. **Multi-grouped attention (MGA)** is a hybrid between **multi-head attention (MHA)** and sparse attention. Instead of each attention head attending to all tokens, MGA groups multiple heads together to enable more efficient computation. In MGA, the heads are not separated but grouped into specific clusters and process a group of characters. This makes it possible to reduce computational costs, is more flexible for sparse attention forms, and makes it possible to speed up training and reasoning. Another popular alternative is **multi-head latent attention (MLA)**, which is used in modern LLMs. In standard MHA, we explicitly compute attention for all heads. In MLA, we use latent heads that indirectly encode relationships between tokens without the need for a full pairwise computation of attention. In this way, the model has better generalization by learning a compressed representation without sacrificing accuracy. This requires less attention during inference and saves memory.

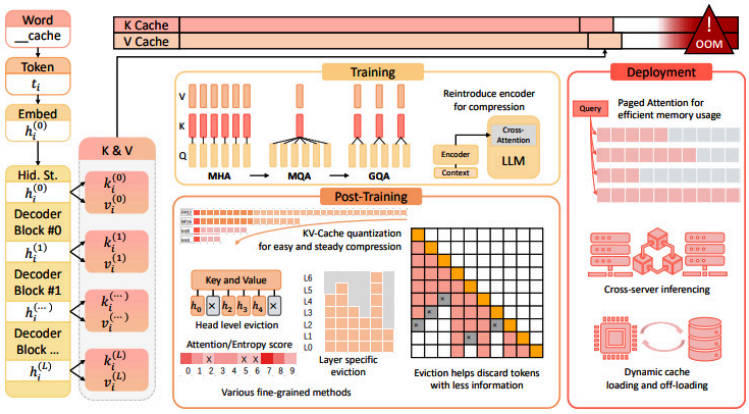


Figure 10.39 – Overview of methods for speeding inference (https://arxiv.org/pdf/2407.18003)

These techniques, as illustrated in *Figure 10.39*, demonstrate how inference efficiency can be improved across multiple stages—compression, caching, and memory optimization. With this foundation, we can now explore how such optimizations are applied in real-world deployment scenarios.

Data, pipeline, and tensor parallelism

Another way to make training more efficient is to parallelize it. **Model parallelism** for a neural network is to distribute the model across multiple devices (such as GPUs or TPUs) to overcome memory and computation limitations. While this can be useful to speed up training, in other cases, it is necessary because the model is too large to fit on a single device. There are several ways to parallelize a model, as we will see next:

- **Data parallelism** is considered the simplest approach, in which replicas of the model are distributed across multiple computing devices (e.g., GPUs, TPUs, or even different machines), and different subsets of the training dataset are fed into each replica. During training, averaging of the gradients of the various GPUs is conducted; this is used for model updates. Then, each model is replicated across workers (GPUs/TPUs), and the input data batch is split into mini-batches assigned to different workers. During the forward pass, each worker computes predictions and losses for its mini-batch. Subsequently, each worker calculates gradients for its assigned data. These gradients are aggregated either by averaging or using a more complex method, and the aggregated gradients are used to update all model replicas, ensuring synchronization across workers. Data parallelism can be implemented in several ways, the most common being synchronous data parallelism, in which all devices compute the gradient before synchronization. Once all gradients are available, averaging is conducted. Although this approach ensures that there is consistency, a worker can slow down the training. To overcome this, we have asynchronous data parallelism, where each device conducts the local model update independently, at the risk of introducing stale gradients (outdated updates). An intermediate approach (stale-sync data parallelism) is also available, where workers perform multiple local updates before synchronizing with others. Data parallelism can also be centralized with a central server or decentralized with the various workers exchanging gradients in a ring topology. Data parallelism allows the workload to be distributed among different devices, increasing the speed of training, scales well when you have several devices, is not complex to implement, and is efficient because the model stays on the various devices and is not swapped. On the other hand, gradient synchronization can be slow due to communication overhead, especially if communication is inefficient. Variations in device speed, such as using different hardware or GPU versions, can further exacerbate this issue. Additionally, large batch sizes may cause convergence problems, and managing synchronization becomes increasingly complex as the number of devices grows.

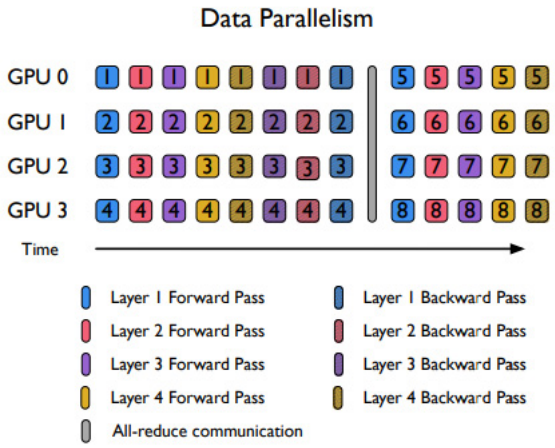


Figure 10.40 – Processing of mini-batches over time in data parallelism. Each GPU has a copy of all the layers (shown in different colors) and different mini-batches (numbered) are processed by different GPUs (https://arxiv.org/pdf/2111.04949)

- **Pipeline parallelism** is a distributed training technique where different layers of a deep learning model are assigned to different devices (e.g., GPUs or TPUs), and mini-batches are processed sequentially

through the pipeline. This technique helps in training extremely large models that do not fit into a single device's memory. Pipeline parallelism is commonly used in transformer models such as GPT-3, GPT-4, LLaMA, and DeepSeek, where model sizes exceed the memory capacity of a single GPU. The model is divided into multiple stages, where each stage represents a subset of consecutive layers and is assigned to a different GPU. A batch is split into mini-batches, and a mini-batch is split into micro-batches. One micro-batch is then processed from the first stage and passed to the next. The second micro-batch starts being processed before the first micro-batch has finished all the stages (as soon as the first stage clears, it can start processing the second micro-batch, without the first micro-batch having to pass all the layers, thus allowing the process to be parallelized in an efficient manner). The backward pass follows the same pipeline as the forward pass but in reverse order; the gradient starts from the last stages to the first stages. Once all micro-batches are completed, the model update can be conducted.

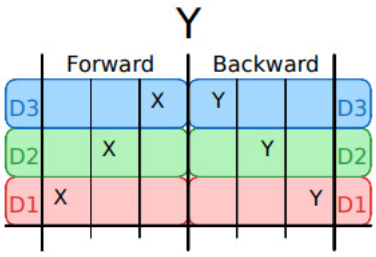


Figure 10.41 – Forward and backward update for a single micro-batch (https://arxiv.org/pdf/2403.03699v1)

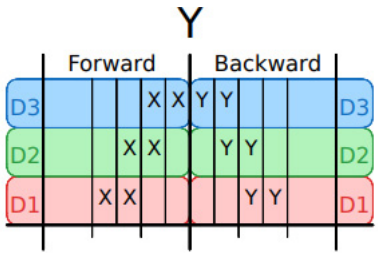


Figure 10.42 – Forward and backward update for two micro-batches in parallel (https://arxiv.org/pdf/2403.03699v1)

Pipeline parallelism can be conducted in different manners such as **one forward, one backward (1F1B)** scheduling, in which each GPU conducts one forward pass and one backward pass at the same time. Alternatively, each device could contain multiple model partitions and thus conduct more flexible scheduling. Pipeline parallelism allows the training of very large models that do not fit into a single GPU, allows better utilization of the various devices (each device constantly processes micro-batches), reduces the risk of memory bottlenecks, and is well adapted to transformers. On the other hand, it is a more complex system, where one has to manage the stages so that some of them do not have more computation-heavy layers and thus become bottlenecks (careful layer partitioning to balance the workload among the various devices). In the first iterations, the system is less efficient as it waits to be filled with micro-batches (the first stage starts working before the other stages), communication is more complex due to gradient aggregation, and there is increased complexity in designing the system.

- **Tensor parallelism** is a model parallelism technique where individual weight tensors (matrices) within a model are split across multiple GPUs. Unlike traditional model parallelism, which assigns entire layers to different GPUs, tensor parallelism breaks down the computations within a single layer and distributes them across multiple devices. This approach is particularly useful for large-scale transformer models where certain operations (such as matrix multiplications in attention layers) require enormous memory and computational power. Instead of computing and storing entire weight matrices on a single GPU, tensor parallelism divides them among multiple GPUs. For example, a fully connected layer applies a weight matrix, *W*, to an input, *X*, to obtain an output, *Y*. If *W* is too large for a single GPU, we can divide it among multiple GPUs. Each GPU will then conduct only part of the computation, producing part of the output, which is then later aggregated. Similarly, during the backward pass, we must then redistribute the gradient computation to allow proper updates of the weights of the various matrices, *W*. Column-wise tensor parallelism is among the most widely used for transformers, where the weight matrix is split column-wise across GPUs, and each GPU then computes part of the output, which is then concatenated. Considering the self-attention mechanism of a model, the query (*Q*), key (*K*), and value (*V*) matrices are split column-wise across multiple GPUs. Each GPU then computes a partial attention score, following which the various results are aggregated across GPUs to reconstruct the finished output. The advantage of this approach is that instead of storing entire weight matrices, each GPU stores only a portion. Also, the multiplication of large matrices can be distributed and thus make the computation faster, making it particularly efficient for large models. On the other hand, there is always the risk of communication overhead (GPUs must frequently exchange partial results, which can slow down training), it can be complex to implement, and it is not worthwhile except for large models.

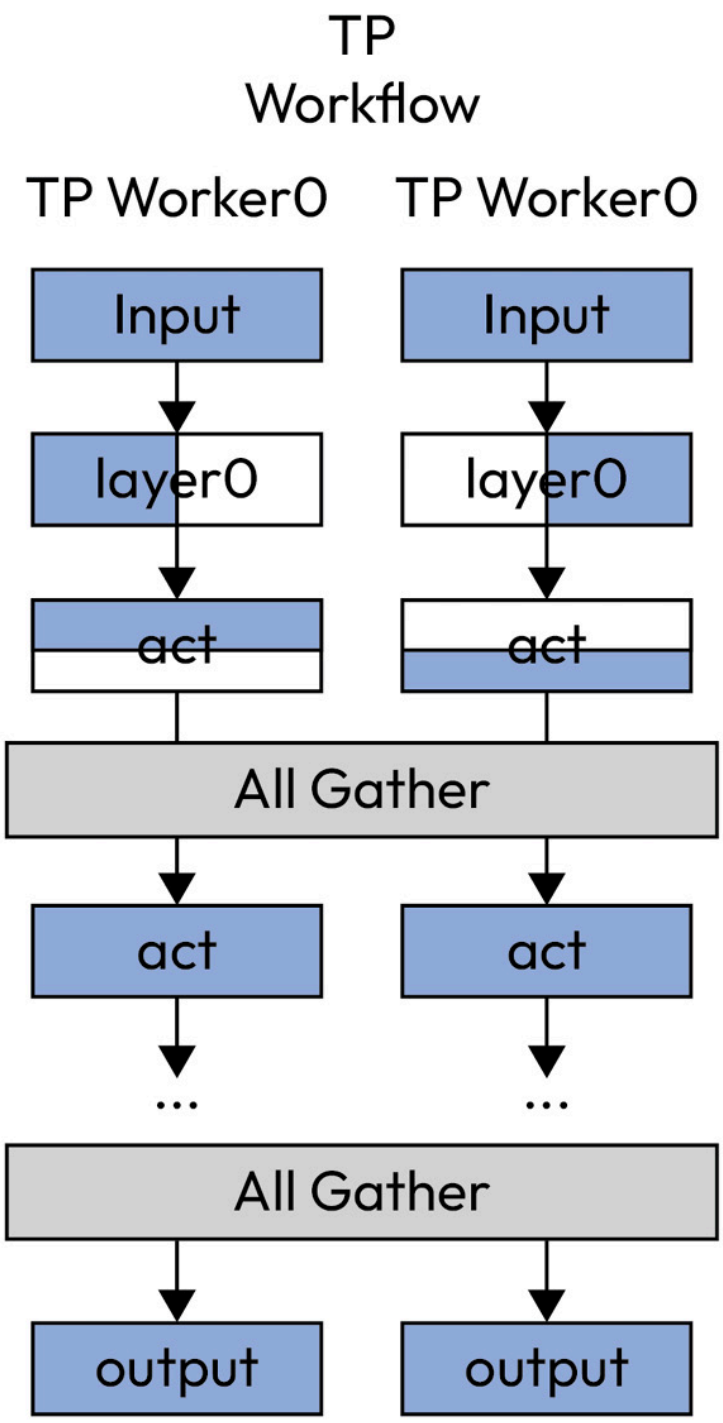


Figure 10.43 – Tensor parallelism (https://arxiv.org/pdf/2311.01635)

The following table compares tensor parallelism, data parallelism, and pipeline parallelism across key dimensions such as memory usage, communication overhead, and complexity:

Feature	Tensor parallelism	Data parallelism	Pipeline parallelism
How it works	Splits individual tensors across GPUs	Replicates full model on each device; splits data	Splits model layers across GPUs

Memory usage	Low (weights are sharded)	High (full model stored on each GPU)	Medium (layers distributed)
Communication overhead	High (frequent cross-GPU communication)	High (gradient synchronization)	Moderate (micro-batch passing)
Best for	Very large models with huge weight matrices	Medium-sized models with large datasets	Deep models such as transformers
Complexity	High	Low	Medium

Table 10.1 – Comparison of tensor, data, and pipeline parallelism in large-scale model training

Hybrid parallelism integrates different types of parallelism trying to optimize training across multiple GPUs. Generally, the various approaches can be combined, although this requires more complexity. For example, data parallelism ensures that GPUs process different batches while model parallelism (tensor or pipeline parallelism) ensures that the model is optimized across multiple GPUs. For example, if the model is too large for a single GPU, we can use model parallelism and split the model across multiple GPUs. We can then use 16 GPUs to split a batch of data across 4 copies of the model.

So far, we have explored how to build a fully working AI-driven Streamlit app that integrates multiple agents and external APIs such as OpenAI. However, when an application moves from development to production, some important challenges need to be taken into account.

Handling errors in production

In this section, we'll explore some of the approaches we can adopt to handle issues that may occur when an application moves from development to production. Typical problems you might encounter include:

- The OpenAI API is temporarily unavailable
- Intermittent network failures or exceeding rate limits
- Incomplete or missing logging system

Let's see how we can mitigate these issues effectively:

- **The OpenAI API is temporarily unavailable**: One simple and effective way to start handling these issues is by wrapping your API calls in **`try`**/**`except`** blocks. Here's an example of how you can handle different types of errors when calling the OpenAI API:

```
try:
     response = client.chat.completions.create(
         model="gpt-4",
         messages=[...],
         timeout=10           # optional timeout
```

```
        )
        return response.choices[0].message.content
    except openai.RateLimitError:
        st.error("Rate limit exceeded. Please try again later.")
    except openai.APIError as e:
        st.error(f"OpenAI API error: {str(e)}")
    except Exception as e:
        st.error(f"Unexpected error: {str(e)}")
```

- **Temporary issues**: When there are intermittent network failures or momentary unavailability of external APIs, instead of immediately failing, the app can retry the operation a few times:

```
import time
 import random
def call_openai_with_retry(prompt, retries=3):
    for i in range(retries):
        try:
            return client.chat.completions.create(
                model="gpt-4",
                messages=[ {"role": "user", "content": prompt}]
            )
       except openai.APIError:
            wait = 2 ** i + random.random()
            time.sleep(wait)
   st.error("Failed after multiple retries.")    return None
```

- **Logging system**: Using **`st.write()`** is fine for quick debugging, but in production, you need a more persistent and structured way to track what's happening in your app.
  A basic logging system helps you record important events and catch errors that may not appear in the UI:

```
import logging
 logging.basicConfig(level=logging.INFO)
 logger = logging.getLogger(__name__)
try:
    logger.info("Calling OpenAI API")
    response = client.chat.completions.create(...)
except Exception as e:
    logger.exception("API call failed")
    st.error("Something went wrong.")
```

Security considerations for production

Applications deployed in production often involve API keys and potentially sensitive user data, so security must be carefully addressed from the beginning.

One of the most fundamental practices is to avoid hardcoding credentials such as API keys directly into the source code. Instead, credentials should be managed securely using environment variables or a dedicated secrets management system.

Security in production typically involves three key areas:

- Managing secrets
- Data exposure prevention
- Securing your deployment environment

Let's discuss these next.

Managing secrets in production

There are two common ways to securely manage secrets in production environments:

- **Using `st.secrets`**: This is ideal for applications deployed on Streamlit Cloud
- **Using environment variables**: This is recommended for Docker containers or local server deployments

Both approaches allow you to keep sensitive information out of your source code, but the right choice depends on your deployment context.

Here are some examples for each method:

- **Using `st.secrets`**: When using Streamlit, create a **`.streamlit/secrets.toml`** file that lets you define secrets into it. Here is an example:

  ```
  [general]
   openai_api_key = "application-api-key"
  ```

  Access it in your code like this:

  ```
  import openai
   openai.api_key = st.secrets["general"]["openai_api_key"]
  ```

- **Using environment variables**: For Dockerization or local deployments, it is recommended to store secrets as environment variables, keeping them separate from the source code. To use environment variables, you must define them in your terminal or deployment environment before running your application.
  For example, in a Unix-based terminal (Linux, macOS, or WSL), you can define the variable like this:

  ```
  export OPENAI_API_KEY="your-api-key"
  ```

  Then, in your Python code, access the variable as follows:

  ```
  import os
  openai.api_key = os.getenv("OPENAI_API_KEY")
  ```

  The **`export`** command sets an environment variable only for the current terminal session. This means it will remain active only until you close the terminal. To launch your app using the variable, you must run it in the same shell session:

  ```
  export OPENAI_API_KEY="your-api-key"
   streamlit run app.py
  ```

  To make the variable available every time you open a terminal, you can add it to your shell's startup file.
  On Linux, this file is usually called **`~/.bash_profile`**.
  These are called initialization scripts. They are automatically executed every time you start a new terminal session and are used to configure the shell environment, including setting environment variables, aliases, and paths.

To add the API key to **`~/.bash_profile`**, open the terminal and run the following:

```
nano ~/.bashrc
```

Then, save and close it. From now on, your app will automatically find the API key every time it is launched from a new terminal session.

Data exposure prevention

In production, one of the most overlooked security risks is the unintentional exposure of sensitive data through logging, error messages, or misconfigured URLs.

While logging is essential for debugging and observability, it can easily become a liability if secrets, tokens, or user data are captured without proper filtering.

Here are a few best practices to minimize the risk:

- **Avoid logging secrets**: Never print API keys, access tokens, or passwords to logs, even in debug mode. This applies to both client-side and server-side logs.
- **Sanitize user data**: If your application logs inputs or error traces that include user-provided data (e.g., form submissions, headers, and payloads), be sure to mask or strip sensitive fields (such as email addresses, credit card numbers, or personal identifiers).
- **Configure logging levels appropriately**: Use different log levels (e.g., **`INFO`**, **`WARNING`**, **`ERROR`**, or **`DEBUG`**) and restrict debug-level logs in production. Enable only what is necessary to diagnose issues without overexposing internals.
- **Handle errors**: Avoid sending raw stack traces or system error messages directly to users. These can leak details about your backend, framework, or database.

Preventing data exposure is about designing systems that assume that secrets and user data must always be protected, even in edge cases or failures.

Securing your deployment environment

Even if your code avoids data exposure and your secrets are properly managed, your application can still be vulnerable if the environment in which it runs is misconfigured.

For example, in modern workflows, containerization is one of the most common ways to package and deploy applications. In fact, containers offer portability and consistency across environments, but they also introduce specific security risks.

A wrong or poor Dockerfile configuration can introduce multiple vulnerabilities, such as the following:

- Increased exposure to known exploits if the image includes unnecessary packages or tools
- Credential leaks if secrets are stored directly in the image
- Privilege escalation if the container runs as the root user

- Unsafe access to host resources if volumes are not properly restricted

To mitigate these risks, it is important to follow a set of container security best practices. Let's look at a few simple guidelines to make your Docker-based deployment more secure and production-ready:

- **Bloated images** = **more attack surface**: Using a full **`python:3.11`** image instead of **`python:3.11-slim`** can include dozens of unnecessary system tools. If any of these have known vulnerabilities, they become an unintentional attack, even if your app doesn't use them.
- **Secrets in the image** = **easy leaks**: Hardcoding API keys or copying **`.env`** files into the Docker image allows anyone with access to the image to extract and appropriate them.
- **Running as `root`** = **dangerous escalation**: If no user is specified, containers run as **`root`**, and combined with an exploit in a Python dependency, this could give an attacker full control of the container and possibly the host too.
- **Unsafe volume mounting** = **host access**: Mounting critical paths gives containers access to critical host files including SSH keys and system configuration. If the container is compromised, the host is compromised as well. Examples of critical paths include the following:
  - **`/`**: Root of the host filesystem. Grants full access to the entire filesystem of the host, including sensitive system directories, user data, and configuration files.
  - **`/etc`**: System configuration directory. Contains critical configuration files, including **`/etc/passwd`**, **`/etc/shadow`**, network settings, and user permissions. Exposing this can allow manipulation of how the host system behaves.
  - **`/var/run/docker.sock`**: Docker daemon socket. Gives the container direct control over the Docker engine running on the host. This lets the container start, stop, and manage other containers, including mounting volumes and executing code on the host.

Here is an example of a minimal and secure Dockerfile:

```
# Minimal python image
 FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
 RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# Create and use a non-root user through the keyword USER
 RUN useradd -m appuser
 USER appuser
CMD ["streamlit", "run", "app.py"]
```

To inject secrets securely at runtime, use environment variables passed with **`docker run`** or use secret management tools such as **Docker secrets**:

```
# Runtime execution
 docker run -e OPENAI_API_KEY="your-api-key" my-streamlit-app
```

MLOPs and LLMOPs are important concepts for anyone who wants to use an ML model or LLM in production. In the next section, we will discuss other important concepts in production deployment, such as asyn-

chronous programming, which allows us to handle multiple concurrent user requests.

Asynchronous programming

So far, you've seen examples where tasks are executed one after the other. But what if some tasks don't need to block the flow of the entire program while waiting? That's where asynchronous programming comes in.

Asynchronous programming allows tasks to cooperatively share the CPU. Instead of each task waiting for the previous one to finish, tasks can voluntarily pause and let others run, making better use of the single processor's time. This does not imply simultaneous execution; instead, it indicates a smart interleaving of their operations.; this is especially useful when waiting for things such as I/O operations.

Think of it as multiple conversations happening with one person switching between them, efficiently and politely. In Python, this is achieved using the **`asyncio`** module, which supports cooperative multitasking on a single CPU.

As you'll see in the comparison table, asynchronous code is different from using threads or multiple processes. It runs on just one core, but it can still feel fast, especially when dealing with many I/O-bound tasks.

Python module	Number of CPUs	Task switching style	Switching decision
`asyncio`	Single	Cooperative multitasking	Tasks yield control voluntarily through the **`await`** keyword
`threading`	Single	Preemptive multitasking	OS decides when to switch threads
`multiprocessing`	Multiple	Preemptive multitasking	Separate processes run independently, but on the same machine; the OS decides when to switch

Table 10.2 – Concurrency mechanisms in Python: differences between asyncio, threading, and multiprocessing

Concurrency is particularly useful in two types of scenarios: when a program is waiting for responses from external systems (I/O-bound), and when it is handling a high computational workload (CPU-bound).

In I/O-bound situations, a script spends most of its time waiting for data to arrive from a source, such as a filesystem, a network connection, a database, or an API. During this time, the CPU is often idle, making it a perfect opportunity to run other tasks concurrently.

In contrast, CPU-bound tasks keep the processor fully occupied with calculations such as rendering images, parsing large datasets, or performing

cryptographic operations. In these cases, concurrency helps by distributing the workload across multiple CPU cores, enabling true parallel execution. This form of concurrency (better described as parallelism) can significantly reduce total processing time for heavy computations.

Type of task	Main limitation	Examples	Concurrency benefit	Execution style
I/O-bound	Slow external systems	Reading files, API requests, database queries	Keeps CPU busy while waiting for I/O	Cooperative (`asyncio`)
CPU-bound	Intensive computation	Data crunching, image processing, encryption	Distributes load across multiple cores for real parallelism	Preemptive (`threading`, `multiprocessing`)

Table 10.3 – I/O-bound vs. CPU-bound: task types and optimal concurrency models

The following diagram illustrates how task execution differs between synchronous and asynchronous when dealing with I/O-bound operations.

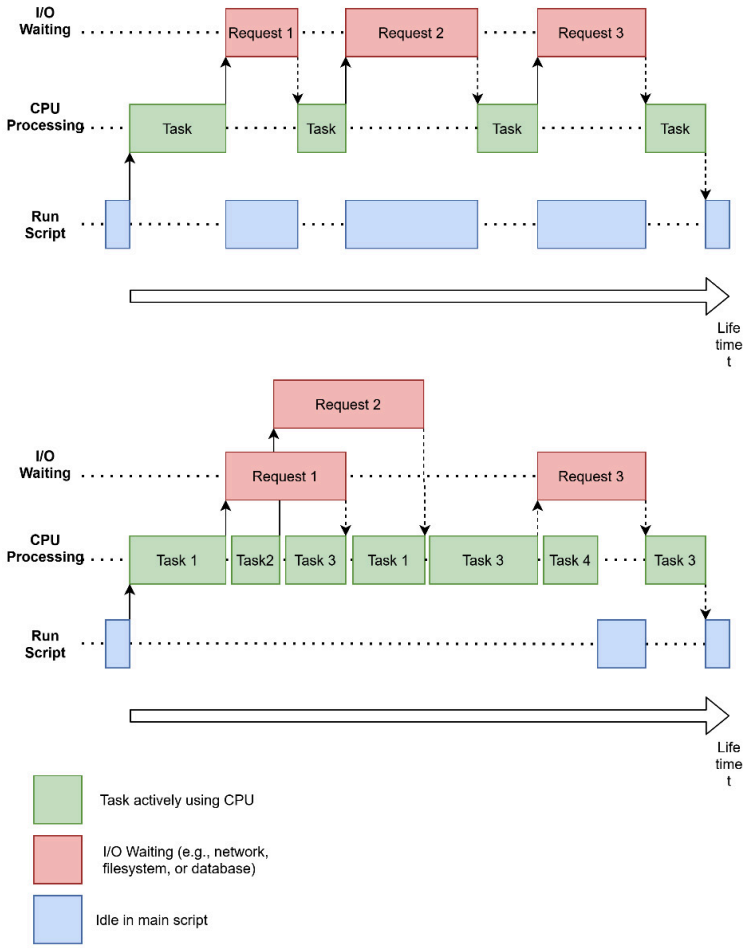


Figure 10.44 – Comparison of blocking vs non-blocking I/O execution

In the first row, each request blocks the CPU until the I/O completes. In the second row (async), the CPU switches between tasks during I/O wait times, improving efficiency on a single core and minimizing the idle time.

When multiple I/O-bound requests arrive in sequence, using a single thread to handle each of them one after the other would block the program during I/O waits.

To improve responsiveness, the **`threading`** module can be used to delegate each request to a separate thread.

In the following diagram, each incoming request is assigned to one of four worker threads. The actual workload (T1, T2, T3, ...) represents short bursts of CPU activity interleaved with I/O waits:

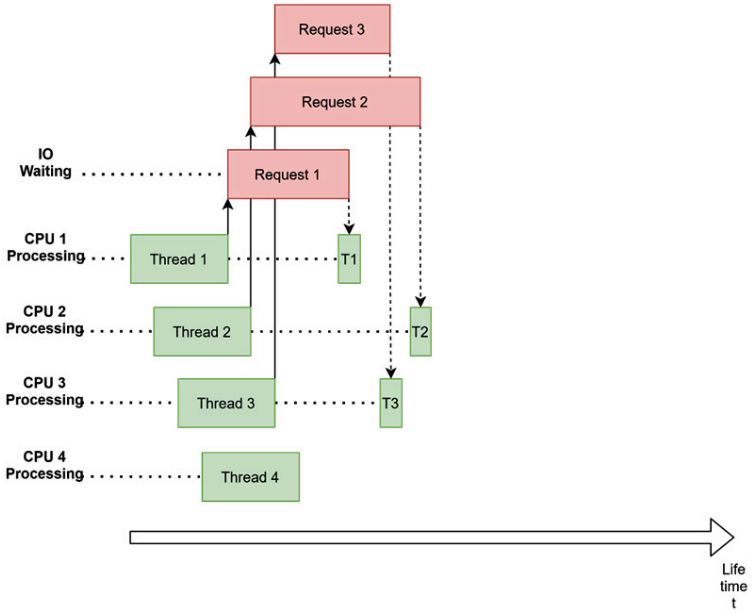


Figure 10.45 – Concurrent request handling with four worker threads and interleaved CPU/I/O workloads

This pattern is useful when your program must remain responsive while interacting with slow external systems such as APIs, databases, filesystems, or even GUIs.

Asynchronous programming is a type of parallel programming that allows programs to perform tasks concurrently, without blocking the main execution thread. For example, when we have multiple users interacting with the system at the same time, we will have more tasks to handle at the same time, with some tasks taking more time by blocking our agent. In traditional synchronous programming, tasks are executed one after the other, where each task must wait for the previous one to finish before it can begin; tasks are executed sequentially in the order in which they are written. Each task must complete fully before the next begins, which can lead to delays if a task involves waiting, such as for file I/O or network operations. Asynchronous programming, on the other hand, allows tasks that may block execution to be initiated and handled concurrently. Instead of waiting for a task to finish, the program can move on to other tasks, returning to the blocked task once it is ready. This approach improves efficiency by making better use of system resources, particularly in scenarios involving high-latency operations, such as web requests or database queries, enabling more responsive and performant applications.

There are some key concepts for discussing asynchronous programming:

- **Concurrency**: This refers to the ability to handle different tasks at the same time; however, this does not mean that tasks are handled simultaneously. Tasks are started and completed in overlapping time periods, but not simultaneously.
- **Parallelism**: This refers to the ability to accomplish tasks at exactly the same time, usually by using multiple processors or cores. While concurrency may or may not involve parallelism, parallelism always involves concurrency.
- **Blocking operations**: These are operations that wait for a task to complete before starting a new operation (e.g., reading a file from disk before starting to process text).
- **Non-blocking operations**: This refers to the ability to start a task and continue the program with other tasks without waiting for the task to complete (making an HTTP request and continuing to generate more text with an LLM while waiting for the response).
- **Callbacks**: These are functions passed as arguments to other functions that are executed when a task completes.
- **Promises and futures**: These are abstractions that represent the eventual result of an asynchronous operation. A promise is a value (or result) that may be unavailable at that time but will be available at some later point. A future is the same thing but is commonly used in languages such as Python and Java.
- **Event loop**: This is the fundamental component of an asynchronous program, where tasks, events, or signals are listed and scheduled for execution when the resources are available. In other words, we use an event loop to allow tasks to run without blocking the main program. The event loop waits for an event to occur and calls an appropriate callback function at this point.
- **Coroutines**: These are special functions that can be paused and then resumed during their execution. In other words, a function can start and then be paused to wait for the result of another task. For example, when we start an analysis of some documents, the function pauses while we conduct an HTTP request to find more information that is needed to accomplish our function. When the results of the HTTP request arrive, the function resumes.

It may seem counterintuitive how asynchronous programming makes code execution faster (after all, no additional resources are being used). I have made extensive changes here for conciseness and clarity. Please confirm whether your intended meaning has been retained. In the synchronous format, she completes each game one at a time before moving to the next. With each move taking her 10 seconds and her opponent 60 seconds, a full game of 30 moves per player (60 moves total) takes 2,100 seconds. Playing all 24 games sequentially requires 50,400 seconds, or roughly 14 hours.

In contrast, the asynchronous format has Judit moving from board to board, making one move per game while each opponent thinks during her rotation. One full round of 24 moves takes 240 seconds, and since each player takes 60 seconds to respond, Judit returns to each board just as the opponent is ready. Over 30 rounds, the entire session lasts only 7,200 seconds, or approximately 2 hours—making asynchronous play significantly more time-efficient.

In async programming, we do exactly the same, the event loop allows us to manage the various tasks in an optimal time management manner. A function that would block other tasks can be optimally blocked when we need to run other tasks, allowing optimized management of the entire program. Here, we do not want to optimize the time of each game but the whole performance.

We can then manage multiple processes at the same time in different ways:

- **Multiple processes**: A process is an independent program in execution. Each process has its own memory, resources, and execution context. In the simplest way, we can manage different processes at the same time (for example, several players playing the 24 games is a simple example of multiple processes occurring at the same time during performance). In the case of programming, this means that different scripts or processes can run at the same time (e.g., four functions and each of them runs on a different CPU). However, this approach is very inefficient.
- **Multiple threads**: This is a variation of the previous approach. A thread is the smallest unit of execution within a process. Multiple threads can be within the same process and share the same memory, but each thread has its own execution stack. In this case, several threads are executed at the same time.
- **Asynchronous programming**: In this case, we have a single process and a single thread but conduct several things at the same time. In Python, **`asyncio`** does exactly this by exploiting coroutines and futures to simplify asynchronous code.

Asynchronous programming, therefore, improves performance when some tasks are time-consuming and can block the execution of a program. In this way, the system can continue executing other tasks while it waits for them to complete. It also allows better utilization of system resources (for example, while waiting for a network request, the program can perform calculations or handle other requests). Asynchronous programming also helps to achieve systems that are more scalable and can handle multiple requests in parallel, reducing the number of threads.

asyncio

asyncio is a Python library that allows you to write concurrent code using the **`async`**/**`await`** syntax. It provides a framework for running asynchronous operations, without relying on multithreading or multiprocessing. The heart of **`asyncio`** is the event loop, which schedules and executes asynchronous tasks (called coroutines) in the background. A coroutine is similar to a generator in Python: it can pause execution and let other tasks run and then resume later. It is the event loop that tracks the state of these coroutines and their results, which are presented as **`futures`**.

Here is a basic example of a coroutine:

```
import asyncio
async def my_coroutine():
    print("Hello, world!")
# Create an event loop and run the coroutine
asyncio.run(my_coroutine())
```

While this code shows how to define and run a coroutine by using the event loop, it does not yet take advantage of concurrent execution. In fact, to execute multiple asynchronous tasks concurrently, we can use either **`asyncio.gather()`** or **`asyncio.create_task()`**.

While **`gather()`** is useful when you want to run several coroutines and wait for all of them to finish together, **`create_task()`** provides more flexibility. It allows you to launch coroutines in the background and decide when (or whether) to await their results later in your program. Let's look at some examples together.

The following example uses **`asyncio.gather()`** to execute multiple coroutines concurrently:

```
async def task1():
    await asyncio.sleep(2)
    print("Task 1 completed!")
async def task2():
    await asyncio.sleep(1)
    print("Task 2 completed!")
async def main():
    await asyncio.gather(task1(), task2())  # Run both tasks concurrently
asyncio.run(main())
```

In this case, both tasks are executed concurrently, and the total execution time will be close to 2 seconds: the time taken by the longest task.

We can achieve the same result using **`asyncio.create_task()`**, which offers more control over task scheduling. Unlike **`asyncio.gather()`**, which groups coroutines and waits for all of them together, **`create_task()`** lets us launch coroutines individually and decide when to await their results. This is particularly useful when we want to run background tasks while doing other work.

Here is the same example rewritten with **`create_task()`**:

```
import asyncio
async def task1():
    await asyncio.sleep(2)
    print("Task 1 completed!")
async def task2():
    await asyncio.sleep(1)
    print("Task 2 completed!")
async def main():
    t1 = asyncio.create_task(task1())
    t2 = asyncio.create_task(task2())
    # Both tasks start running in the background immediately
    await t1
    await t2
asyncio.run(main())
```

Each call to **`create_task()`** returns a **`Task`** object, which represents the running coroutine and can be awaited, cancelled, or monitored.

The result is the same: both tasks run concurrently, and the output is printed once each finish. However, with **`create_task()`**, we gain more flexibility.

For example, we can start several background tasks and continue executing other logic in **`main()`**. Then, we can await only the results we need at a specific point in the workflow. This flexibility makes **`create_task()`** especially useful in complex workflows where not all tasks are equally important or time-sensitive.

To better understand the real-world impact of asynchronous programming, let's compare an example of synchronous versus asynchronous execution. Specifically, we will simulate fetching data from a website using HTTP requests by using Python **`requests`** library. This will highlight how asynchronous code can significantly improve performance when dealing with I/O-bound tasks such as network calls.

Here is the synchronous code:

```
import requests
import time
def fetch_url(url):
    response = requests.get(url)
    return f"Fetched {url}"
def sync_fetch():
    urls = ['https://httpbin.org/get'] * 5  # Simulating 5 requests to the same URL
    results = [fetch_url(url) for url in urls]
    for result in results:
        print(result)
def main():
    start_time = time.time()
    sync_fetch()
    end_time = time.time()
    print(f"Synchronous version took {end_time - start_time:.4f} seconds")
# Run the synchronous example
main()
```

Here is the asynchronous code:

```
import asyncio
import aiohttp
import time
async def fetch_url(session, url):
    async with session.get(url) as response:
        await response.text()  # Simulate processing the response
        return f"Fetched {url}"
async def async_fetch():
    urls = ['https://httpbin.org/get'] * 5  # Simulating 5 requests to the same URL
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(session, url) for url in urls]
        results = await asyncio.gather(*tasks)
        for result in results:
            print(result)
async def main():
    start_time = time.time()
    await async_fetch()
    end_time = time.time()
    print(f"Asynchronous version took {end_time - start_time:.4f} seconds")
# Directly calling the asynchronous function in Jupyter
await main()
```

The following figures show the output of the synchronous and asynchronous implementations described above, respectively. As we can see, the synchronous version performs the HTTP requests one after the other,

resulting in a longer total execution time. The asynchronous version, on the other hand, sends all requests concurrently, significantly reducing the total time required.



Figure 10.46 – Synchronous result



Figure 10.47 – Asynchronous result

Asynchronous programming and ML

Conjugating asynchronous programming with ML in Python can be a powerful combination. Asynchronous programming can improve performance by allowing non-blocking operations, such as loading large datasets, running hyperparameter tuning, or interacting with APIs. For example, we can see different possibilities:

- **Data loading**: In ML workflows, especially when working with large datasets, loading and preprocessing data can often be a bottleneck. Asynchronous programming can help speed this up by loading different parts of the data concurrently. For example, you can asynchronously load multiple chunks of a dataset while concurrently performing some I/O-bound tasks (such as data augmentation, cleaning, or transformation).
- **Hyperparameter tuning**: The tuning of hyperparameters is one of the most time-consuming and slowest processes, which can benefit from conducting some tasks asynchronously. For example, when performing a grid or random search on hyperparameters, different configurations can be evaluated simultaneously rather than sequentially.
- **Asynchronous inference**: You can use asynchronous programming to create a non-blocking API to serve trained ML models. This is especially useful when deploying a model for real-time inference and wanting to handle multiple queries simultaneously.
- **Model training**: Although training is usually conducted on different GPUs/CPUs in parallel, asynchronous scheduling can be conjugated to allow better loading and preprocessing of data while training appears in parallel. This is particularly useful when we have different data to retrieve.

We can observe a classic example of hyperparameter tuning. In this simple example with the classic Iris dataset and a simple model, we'll show how using **`asyncio`** saves some time.

Here is the synchronous code:

```
import time
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
def train_and_evaluate_model(n_estimators, max_depth, min_samples_split, min_sample
    # Load dataset
    data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, tes
        # Initialize and train the model
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf
    )
    model.fit(X_train, y_train)
     # Evaluate the model
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
        return (n_estimators, max_depth, min_samples_split, min_samples_leaf, accur
def tune_hyperparameters():
    n_estimators_values = [10, 50, 100, 150, 200]  # Hyperparameter values to tune
    max_depth_values = [5, 10, None]
    min_samples_split_values = [2, 5]
    min_samples_leaf_values = [1, 2, 4]
    results = []
    for n_estimators in n_estimators_values:
        for max_depth in max_depth_values:
            for min_samples_split in min_samples_split_values:
                for min_samples_leaf in min_samples_leaf_values:
                    results.append(train_and_evaluate_model(n_estimators, max_depth
        # Find the best hyperparameters and accuracy
    best_params = max(results, key=lambda x: x[4])
    print(f"Best hyperparameters: {best_params[:4]} with accuracy: {best_params[4]:
# Measure time for synchronous execution
start_time = time.time()
tune_hyperparameters()
end_time = time.time()
print(f"Synchronous version took {end_time - start_time:.4f} seconds")
```

In the preceding script, we run an ML model and search for the best parameters. This script shows how even a small model takes a lot of time to be executed.

```
Best hyperparameters: (10, 5, 2, 1) with accuracy: 1.0000
Synchronous version took 21.7035 seconds
```

Figure 10.48 – Synchronous result

Here is the asynchronous code:

```
import asyncio
import time
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
async def train_and_evaluate_model(n_estimators, max_depth, min_samples_split, min_
    # Load dataset
    data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, tes
    # Initialize and train the model
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf
    )
```

```
    model.fit(X_train, y_train)
    # Evaluate the model
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    return (n_estimators, max_depth, min_samples_split, min_samples_leaf, accuracy)
async def tune_hyperparameters():
    n_estimators_values = [10, 50, 100, 150, 200]  # Hyperparameter values to tune
    max_depth_values = [5, 10, None]
    min_samples_split_values = [2, 5]
    min_samples_leaf_values = [1, 2, 4]
    tasks = []
    for n_estimators in n_estimators_values:
        for max_depth in max_depth_values:
            for min_samples_split in min_samples_split_values:
                for min_samples_leaf in min_samples_leaf_values:
                    tasks.append(train_and_evaluate_model(n_estimators, max_depth,
        results = await asyncio.gather(*tasks)
        # Find the best hyperparameters and accuracy
    best_params = max(results, key=lambda x: x[4])
    print(f"Best hyperparameters: {best_params[:4]} with accuracy: {best_params[4]:
# Measure time for asynchronous execution
start_time = time.time()
await tune_hyperparameters()
end_time = time.time()
print(f"Asynchronous version took {end_time - start_time:.4f} seconds")
```

In this case, we trained the same model using asynchronous programming. This approach allowed us to save time and thus reduce the execution time.

```
Best hyperparameters: (10, 5, 2, 1) with accuracy: 1.0000
Synchronous version took 21.7035 seconds
```

Figure 10.49 – Asynchronous result

This can also be applied to an LLM as an agent. Traditionally, function calls block LLM inference, making the process inefficient as each function call must complete before moving to the next. Some authors propose instead to implement an async approach even with LLMs (or generate tokens and execute function calls concurrently) when tools are connected like in agents. For example, one can consider interruptible LLM decoding, where the function executor notifies the LLM asynchronously, allowing it to continue generating tokens while waiting for function call results. The purpose of this approach is to reduce latency by conducting an overlap of function execution and token generation.

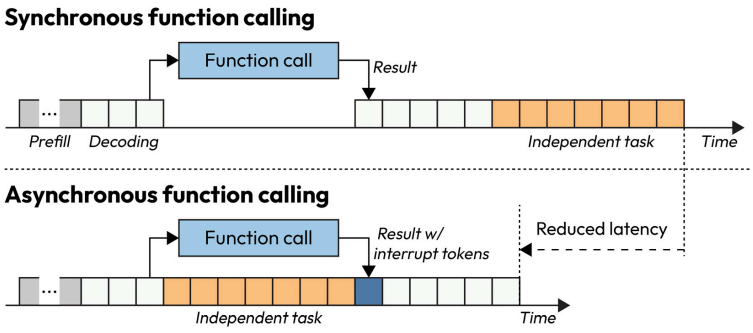


Figure 10.50 – Synchronous vs. asynchronous function calling (https://arxiv.org/pdf/2412.07017)

So, in theory, we can have three approaches for an LLM agent:

- **Synchronous LLM function calling**: Each function is executed one after the other. An LLM must wait for each function to complete before it can continue with the next one. This approach is the simplest, but it adds latency to the system since it must wait for each operation to finish (e.g., reading HTML, reading XLS files, generating tokens, etc.) before it can continue. This leads to high inefficiency, especially if there are many functions or some functions lose a lot of time.
- **Synchronous LLM function calling with parallel optimization**: This process tries to optimize each task in parallel (e.g., reading HTML, reading XLS, and reading text simultaneously), but each task still blocks the next one. The advantage over the previous approach is that each function can be conducted concurrently, with an increase in speed over the previous one. Synchronization is required to conduct the tasks in the right order. Although the tasks are optimized, they are still synchronous, so we have to wait for a function to finish before completing some tasks.
- **Asynchronous LLM function calling**: In this approach, tasks are executed asynchronously, meaning that functions do not block one another. The system can read HTML, read XLS, and read text while simultaneously performing other operations (such as summarizing or saving data). This leads to a noticeable improvement in latency, improving the use of resources. The system ensures that dependent tasks (e.g., summarizing and saving PDFs) are only performed once the necessary data (e.g., reading text) is available. Dependencies are managed dynamically without halting other operations. Multiprocessing parallelization (the previous approach) creates different processes or threads in order to handle tasks concurrently, thus allocating resources and memory. This leads to more resource consumption than in an asynchronous version, and consumption can explode depending on how many functions we have. Also, this approach is more scalable.

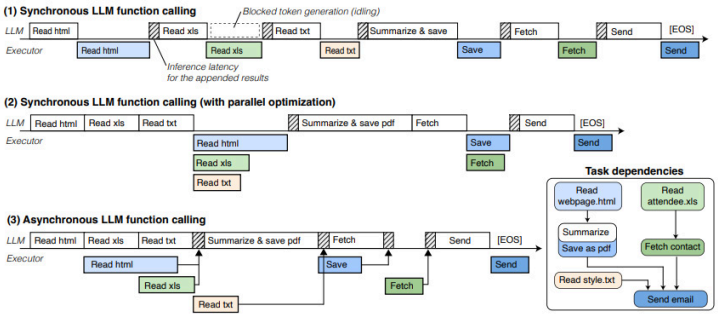


Figure 10.51 – Comparison of LLM-executor interactions (https://arxiv.org/pdf/2412.07017)

Once we have made our system (our application) efficient, it should be placed in isolation to avoid external problems. In the next section, we will explain in detail exactly how Docker allows us to do this.

Docker

Docker is an open source platform that enables developers and system administrators to create, deploy, and run applications in containers. Containers allow software to be packaged along with all its dependencies (such as libraries, configurations, etc.) and run consistently across different environments, whether it's a developer's laptop, a test server, or a production machine.

Containers can then be viewed as virtual machines, allowing for reduced overhead and better utilization of resources and the system itself (especially if we have to use a single model on several systems). The idea is that our software (of which our model or an LLM plus agents is a component) can run in isolation to prevent problems from arising that impact its execution and performance. The use of virtual machines is an example of how a system can run in a guest **operating system** (**OS**) and use resources. Optimizing a system for a guest OS, however, requires considerable resources. Containers try to reduce resource consumption and overhead in order to run the application. Containers offer a way to package an application to make it abstract from the environment in which it runs. This decoupling then allows a container to run in any target environment, predictably and isolated from other applications. At the same time, the container provides the ability to control our environment in a granular manner. Docker containers are lightweight and portable and ensure that the application behaves the same way everywhere. Given these benefits, Docker containers have been adopted by many companies.

Docker is based on a few main concepts:

- **Containers**: These are the basic units of Docker and contain an application and its dependencies in a single package that can be easily moved between environments. A container also contains the OS kernels, to reduce the resources needed. Unlike a virtual machine that contains the entire OS, Docker containers contain only the information needed to run the application. This makes Docker containers much faster and more efficient to run.
- **Images**: An image is a read-only template used to create containers. It contains the application code, runtime, libraries, and environment variables. Docker images contain the blueprint of the application – all the information to be able to execute a code. There are many ready-made images in Docker Hub that can be used to efficiently create containers and reduce the need to start from scratch.
- **Docker Engine**: This is the component responsible for managing and running containers (runtime environment for Docker). Docker Engine runs on both Linux and Windows OSs.
- **Dockerfile**: A Dockerfile is a script containing instructions on how to build a Docker image. This file specifies which base image to use, how to install dependencies, environment configurations, and other details.
- **Docker Compose**: This is a tool for defining and running multi-container Docker applications.

Docker containers thus have a number of advantages:

- **Portability**: Docker containers encapsulate an application and its dependencies in a single, portable unit. In this way, the system abstracts away differences between environments, making it more reliable and consistent in deployment.
- **Efficiency**: The system is more efficient compared to traditional virtual machines. By using only the kernel, the system uses far fewer resources, thus making it easier to deploy and more scalable. Docker integrates well with other orchestration tools, such as Kubernetes and Docker Swarm, making it easier to scale an application both horizontally (more containers) and vertically (increasing the resources available to containers).

- **Isolation**: Docker provides strong isolation between containers, allowing them to run independently and not interfere with each other, thus improving security and avoiding conflicts between different applications.
- **Version control and reproducibility**: A container allows you to store, share, and deploy specific versions of an application, ensuring that a single version is used in different environments, thus improving reproducibility.

Like any system, there are also disadvantages:

- **Security concerns**: It can introduce some vulnerabilities, especially if you're not shrewd and handy with Docker. It has a certain learning curve, especially if you want to use the system efficiently.
- **Data management**: Containers are ephemeral by design, meaning that any data inside a container will be lost if the container is destroyed. Although there are solutions to this problem, it requires more complexity than traditional systems.
- **Complexity**: Docker makes deploying and managing individual containers easy; scaling and orchestrating large numbers of containers across many nodes can become complex. Docker's networking model, while flexible, can be difficult to set up and manage, particularly when containers are spread across multiple hosts. In addition, complexities increase if there are several containers and associated tools. Also, the OS kernel is limited, making debugging and implementing certain features more complex.

While Docker containers offer many advantages such as portability, efficiency, isolation, and scalability, they also come with challenges, especially related to security, complexity, and data management.

Sometimes, our system can be particularly complex and have more than one container; in the following subsection, we will discuss Kubernetes, which allows us to orchestrate multiple containers.

Kubernetes

Kubernetes is an open source container orchestration platform that automates the deployment, scaling, management, and operation of containerized applications. It manages and orchestrates containers in production environments.

In Kubernetes, a Pod is a group of one or more containers that are tied together and share resources such as networks and storage. The containers in a Pod are always deployed together and share the same environment. A Service is an abstraction that defines a logical set of Pods and a policy to access them. Services allow us to manage how our Pods are connected internally or are open to the outside world (during production deployment). A node, on the other hand, is a physical or virtual machine that runs containers in the Kubernetes cluster. Each node in the cluster runs at least one kubelet (the agent that runs containers) and a kube-proxy (networking proxy for managing communication between containers). A group of nodes is called a cluster, and clusters are the backbone of a Kubernetes environment that provides resources such as CPU, memory, and storage to applications. Kubernetes facilitates the deployment and maintenance of containers, allowing easier scaling and production of ap-

plications. It also allows us to better manage sensitive data configuration and data management in general.

Kubernetes is widely used to deploy, manage, and scale microservices-based applications. It is also a popular choice in DevOps practices due to its ability to automate deployments and scale applications.

Docker with ML

Over the years, Docker has been extensively used with ML models, both for running models and for ML-based creations. It allows you to set up a workspace that is ready to code, where all the dependencies needed are managed so that the process of using a model is expedited. Docker also allows for improved reproducibility of models, both for training and inference.

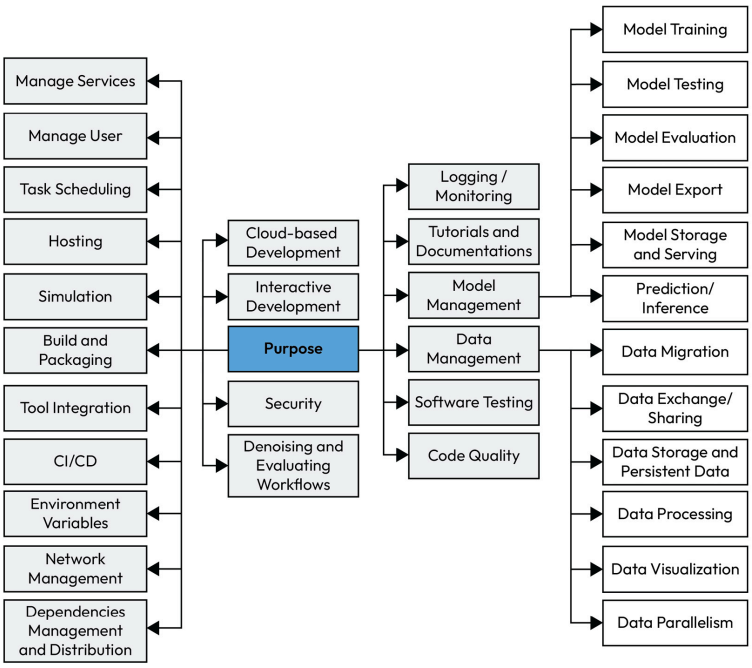


Figure 10.52 – Overview of the purposes of using Docker for ML-based software projects (https://arxiv.org/pdf/2206.00699)

Docker can be used with any ML application, including using LLMs and agents. For example, Ollama has its own Docker image available on Docker Hub, thus making it easy to create applications with LLMs and be able to directly deploy them to a server. Our application can also contain RAG or other components.

Also, as Docker containers are now used in various applications and LLMs are used to generate code, an LLM can be used to address the challenges of environment configuration in software development, particularly when using Docker for containerization. In fact, many software repositories require specific dependencies to function properly, and setting up the environment correctly is error-prone, time-consuming, and difficult for users. Docker allows the process to be more robust and reproducible, but Dockerfiles must be configured manually and can be complex when a project has many dependencies or when the configuration involves multiple steps that need to be executed in a specific order. Therefore, it was proposed to use an LLM to act as an intelligent agent that understands the dependencies and requirements of a repository and can generate a fully automated configuration that works in a Docker con-

tainer. **Repo2Run** is an approach that leverages an LLM as an agent to control the process and ensure that the environment is properly configured before being deployed.

Repo2Run automatically generates Dockerfiles, which are used to configure Docker containers. Dockerfiles contain a set of instructions for setting up a Docker container environment, including installing dependencies and setting up necessary configurations. The system inspects a given Python repository, detects its dependencies (e.g., from files such as **`requirements.txt`** or Pipfile), and then formulates a Dockerfile to recreate the necessary environment. The core innovation in Repo2Run lies in its use of LLMs to drive the configuration process. The LLM intelligently understands the structure of the repository and its dependencies, reducing the need for manual intervention. It automates steps that are traditionally tedious and prone to errors, such as dependency resolution and configuration setup.

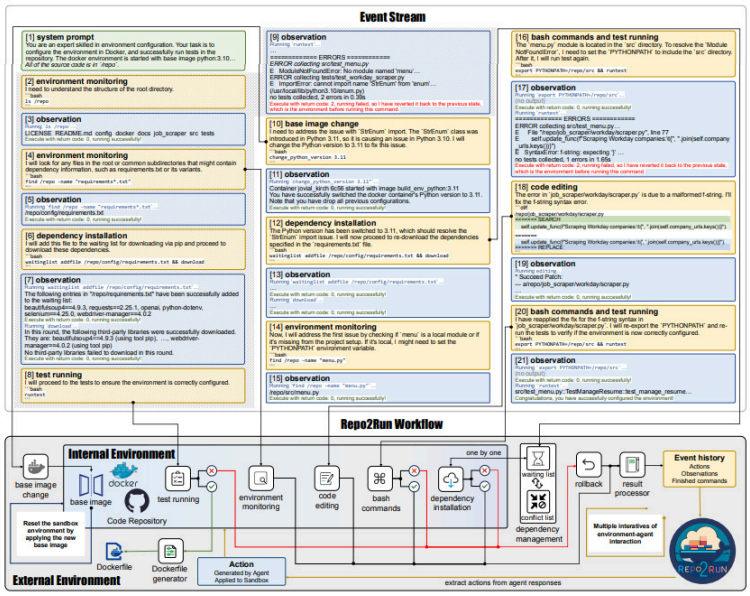


Figure 10.53 – Example process of Repo2Run (https://www.arxiv.org/pdf/2502.13681)

Moving Docker containers to Kubernetes requires a set of configuration files that describe how applications run within Kubernetes clusters (Kubernetes manifests). This migration can be complex, especially for large applications that contain several containers and services. Conducting this process can be error-prone, time-consuming, and difficult to manage, especially for teams without in-depth Kubernetes expertise. Therefore, some works (such as Ueno, 2024) propose to use an LLM to assist in this process and generate the manifest.

The **LLMSecConfig** framework aims to address a critical problem in the security of containerized applications and **container orchestrators (COs)** such as Kubernetes. CO tools are used to manage the deployment, scaling, and networking of containerized applications. However, due to their complexity, many possible misconfigurations can expose security vulnerabilities. For instance, misconfigured access controls, improper resource limitations, or insecure network policies can leave applications open to attacks.

These misconfigurations are common because the process requires a high level of expertise and is manual. **Static analysis tools (SATs)** are used to

detect misconfigurations by analyzing the configuration files of containerized applications, such as Kubernetes YAML files or Dockerfiles. Although SATs are a good solution for detecting vulnerabilities, they lack automation and require manual effort. LLMSecConfig proposes to use RAG and LLMs to find relevant information from external sources to identify misconfigurations. The goal then is to make the process automated, in which vulnerabilities are identified and fixed at the same time while maintaining operational containers.

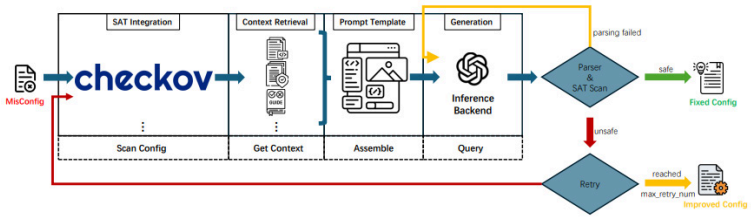


Figure 10.54 – Architecture overview of the LLMSecConfig framework for automated Kubernetes security configuration (https://arxiv.org/pdf/2502.02009)

These approaches show that not only can Docker be used for LLM applications but also, conversely, LLMs can be used to enhance the use of containers, especially when the application goes into production.

Summary

This chapter focused on an important aspect of how we plan a multi-agent system. Whatever form our system takes, it must eventually go into production and be used by users. The experience for users is pivotal to whatever project we have in mind. That is why we started by using Streamlit, a framework that allows us to experiment quickly and get an initial proof of concept. Being able to get a prototype of our system allows us to understand both strengths and weaknesses before investing large resources in scaling. The advantage of Streamlit is that it allows us to analyze both the backend and the frontend, enabling us to interact with an application as if we were one of the users. Streamlit allows us to test what a complete product may look like before we conduct scaling and system optimization.

Obviously, an application will then have to pass this prototype stage to enter production. This step requires that we conduct scaling of our application. LLMs are complex products that need a lot of resources, so during the second half of the chapter, we dealt with all those operations that enable the training and what happens afterward. Although we kept a main focus on LLMs, everything we saw can be useful for any ML application.

In the next and final chapter of the book, we will discuss the perspectives of a field that is constantly evolving. We will discuss some important open questions and some of the future opportunities and developments that the exciting field of agents holds for us.

Further reading

- Hewage, *Machine Learning Operations: A Survey on MLOps Tool Support*, 2022, https://arxiv.org/abs/2202.10169

- Park, *LlamaDuo: LLMOps Pipeline for Seamless Migration from Service LLMs to Small-Scale Local LLMs*, 2024, **https://arxiv.org/abs/2408.13467**
- Zhao, *A Survey of Large Language Models*, 2023, **https://arxiv.org/abs/2303.18223**
- Chang, *A Survey on Evaluation of Large Language Models*, 2023, **https://arxiv.org/abs/2307.03109**
- IBM, *LLM evaluation: Why Testing AI Models Matters*, **https://www.ibm.com/think/insights/llm-evaluation**
- Guo, *Evaluating Large Language Models: A Comprehensive Survey*, 2023, **https://arxiv.org/abs/2310.19736**
- Shi, *Keep the Cost Down: A Review on Methods to Optimize LLM's KV-Cache Consumption*, 2024, **https://arxiv.org/abs/2407.18003**
- Li, *A Survey on Large Language Model Acceleration based on KV Cache Management*, 2024, **https://arxiv.org/abs/2412.19442**
- Zhou, *A Survey on Efficient Inference for Large Language Models*, 2024, **https://arxiv.org/abs/2404.14294**
- Leviathan, *Looking Back at Speculative Decoding*, 2024, **https://research.google/blog/looking-back-at-speculative-decoding/**
- Determined AI, *Tensor Parallelism in Three Levels of Difficulty*, **https://www.determined.ai/blog/tp**
- Geeksforgeeks, *asyncio in Python*, **https://www.geeksforgeeks.org/asyncio-in-python/**
- Gim, *Asynchronous LLM Function Calling*, 2024, **https://arxiv.org/abs/2412.07017**
- *Asynchronous Computation*, **https://d2l.ai/chapter_computational-performance/async-computation.html**
- Openja, *Studying the Practices of Deploying Machine Learning Projects on Docker*, 2022, **https://arxiv.org/abs/2206.00699**
- Muzumdar, *Navigating the Docker Ecosystem: A Comprehensive Taxonomy and Survey*, 2024, **https://arxiv.org/abs/2403.17940**
- Saha, *Evaluation of Docker Containers for Scientific Workloads in the Cloud*, 2019, **https://arxiv.org/abs/1905.08415**
- Ru, *An LLM-based Agent for Reliable Docker Environment Configuration*, 2025, **https://www.arxiv.org/abs/2502.13681**
- Ueno, *Migrating Existing Container Workload to Kubernetes -- LLM Based Approach and Evaluation*, 2024, **https://arxiv.org/abs/2408.11428v1**
- Ye, *LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations*, 2025, **https://arxiv.org/abs/2502.02009**
- Docker, *LLM Everywhere: Docker for Local and Hugging Face Hosting*, **https://www.docker.com/blog/llm-docker-for-local-and-hugging-face-hosting/**

Table of contents collapsed